I2C master slave code

chat.deepseek.com/a/chat/s/f75b8f14-fcd5-4350-a791-c3e6a97446a0

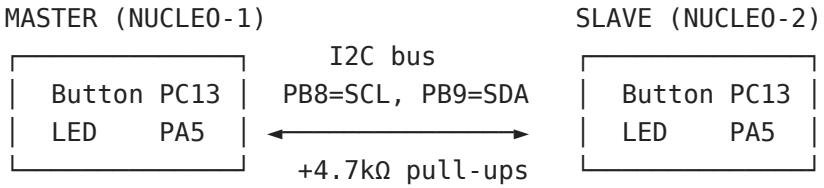
Pasted text.txt Pasted text.txt

I2C master Slave half and full duplex code
explain koro , code work flow and line by line konta keno use hoyese ? example diye bolo, kalk amr lab exam

I2C Two-Way Switch — Full Explanation (Master + Slave)

তোমার lab exam এর জন্য পুরো code টা step-by-step বুঝিয়ে দিচ্ছি। দুইটা board — একটায় **Master** code, আরেকটায় **Slave** code। দুটোতেই একটা button (PC13) আর একটা LED (PA5)।

🎯 প্রথমে বড় ছবিটা (Big Picture)



I2C এ **master সবসময় bus নিয়ন্ত্রণ করে**। তাই master দুই কাজ করে:

1. **WRITE** করে → slave এর কাছে data পাঠায় (`'S'` বা `'K'`)
2. **READ** করে → slave থেকে data নেয় (slave এর button state)

এটাই তোমার "two-way" / "half-duplex" I2C communication। (I2C কখনো **full duplex** না — SDA একটাই wire, একদিকে একসাথে দুই দিকে data যায় না। কিন্তু master একটা session এ write + read দুটোই করতে পারে → এটা "two-way" বা "bidirectional half-duplex")

📋 ৪টা Scenario (কোনটা কী trigger করে)

#	কে trigger করলো	Master কী করে	Slave কী করে	Visual result
1	**Master এর button** চাপ	`'S'` write করে	`blink(5)` — slave এর LED ৫বার	Master little-blink

#	কে trigger করলো	Master কী করে	Slave কী করে	Visual result
2	**Slave এর button** চাপ	periodically read করে `'B'` পায়	শুধু button state return করে	Master blink(5), তারপর `'K'` write → slave little-blink
3	কিছুই না	read করে `0x00` পায়	`0x00` return	দুটোই dark
4	Slave নেই / unplugged	GUARD counter এ আটকে যায়, return 0	—	master hang করে না ✅

🔵 PART 1: MASTER Code — Line by Line

1️⃣ Address macros

```
#define SLAVE_ADDR        0x28
#define BLINK_MS          200
#define SLAVE_BLINK_WAIT  2500
#define GUARD             200000
```

- `SLAVE_ADDR 0x28` → slave এর 7-bit address
- `BLINK_MS 200` → blink এর on/off duration ২০০ms
- `SLAVE_BLINK_WAIT 2500` → master `'S'` পাঠানোর পর 2.5 sec অপেক্ষা করে, যাতে slave এর ৫বার blink শেষ হয় (`5 × 2 × 200ms = 2000ms` + margin)
- `GUARD 200000` → একটা timeout counter। **কেন?** I2C bus এ slave না থাকলে hardware flag কখনো set হবে না → `while(!(flag))` চিরকাল আটকে থাকবে (infinite hang)। GUARD loop এ max 200000 বার চেক করে, শেষ হলে বেরিয়ে যায়।

2️⃣ Register address macros

```
#define RCC_AHB1ENR   (*(volatile unsigned int *)0x40023830)
...
#define I2C1_CR1      (*(volatile unsigned int *)0x40005400)
```

এগুলো STM32F446 এর **memory-mapped register**। Datasheet/Reference Manual থেকে address নেওয়া:

- `RCC_AHB1ENR` → GPIO clock enable
- `RCC_APB1ENR` → I2C1 clock enable
- `GPIOx_MODER/ODR/PUPDR/AFRH/OTYPER` → pin config
- `I2C1_CR1/CR2/DR/SR1/SR2/CCR/TRISE` → I2C control

`volatile` কেন? — compiler যাতে এই read/write optimize না করে (hardware এ value বদলায়, তাই প্রতিবার actual memory থেকে পড়তে হবে)।

3️⃣ Delay & helper

```
static void delay_ms(unsigned int ms) {
    for (unsigned int m = 0; m < ms; m++)
        for (volatile unsigned int i = 0; i < 1300; i++);
}
```

> Software busy-wait delay। ১৩০০ loop ≈ ১ms (১৬০ MHz এ approx)। এটা accurate না, কিন্তু lab এ চলে।

```
static int pressed(void) { return (GPIOC_IDR & (1 << 13)) == 0; }
```

> PC13 input পড়ে। Button এ pull-up, চাপলে pin LOW → `==0` মানে pressed। **BUTTON ACTIVE LOW**।

```
static void blink(int times)  // ২০০ms on/off, `times` বার
static void little_blink(void) // ২বার ১২০ms — "acknowledgement" sign
```

- `blink(5)` = বড় signal ("আমি trigger পেয়েছি")
- `little_blink()` = ছোট acknowledgment ("তোমার message পৌঁছেছে")

4️⃣ `i2c_write()` — সবচেয়ে গুরুত্বপূর্ণ function

```
static int i2c_write(unsigned char addr, unsigned char data) {
```

Return: `1` = slave ACK করেছে (success), `0` = fail (no slave / bus error)। **এটাই তোমার "is slave alive?" check**।

ধাপ ১ — Bus free কিনা দেখা

```
unsigned int g;
g = GUARD; while ((I2C1_SR2 & (1 << 1)) && --g); if (!g) return 0;
```

- `SR2 bit1 = BUSY`। BUSY থাকলে অপেক্ষা করো।
- `--g` দিয়ে count down। GUARD শেষ হলে `g==0` → `return 0`। **Hang হবে না**।

ধাপ ২ — START generate

```
I2C1_CR1 |= (1 << 8);
```

> `CR1 bit8 = START`। Master bus এ START condition দেয় (SCL high থাকতে SDA high→low)।

ধাপ ৩ — SB flag wait (START sent)

```
g = GUARD; while (!(I2C1_SR1 & (1 << 0)) && --g); if (!g) { I2C1_CR1 |= (1<<9);
return 0; }
```

- `SR1 bit0 = SB (Start Bit)`। START পাঠানো হলে hardware SB set করে।
- Timeout হলে `CR1 bit9 = STOP` দিয়ে bus release করে বের হও।

ধাপ ৪ — Slave address + WRITE bit পাঠানো

```
I2C1_DR = (addr << 1) | 0;
```

- `DR` = Data Register। এখানে লিখলে hardware এটা bus এ পাঠায়।
- `(addr << 1)` → 7-bit address কে 8-bit এর উপরের 7 bit এ shift
- `| 0` → LSB = 0 → **WRITE** operation
- অর্থাৎ `0x28 << 1 = 0x50` bus এ যায় → slave চিনে নেয়।

ধাপ ৫ — ACK/NAK wait

```
g = GUARD; while (!(I2C1_SR1 & ((1<<1)|(1<<10))) && --g);
if (I2C1_SR1 & (1 << 10)) { I2C1_SR1 &= ~(1<<10); I2C1_CR1 |= (1<<9); return 0; }
if (!(I2C1_SR1 & (1 << 1))) { I2C1_CR1 |= (1<<9); return 0; }
```

- `SR1 bit1 = ADDR` → address match হয়েছে (slave ACK দিয়েছে)
- `SR1 bit10 = AF (Acknowledge Failure)` → slave ACK দেয়নি (slave নেই!)
- দুটা মধ্যে একটা set হওয়া পর্যন্ত wait।
- **AF set?** → clear করে STOP দিয়ে return 0 (slave নেই)
- **ADDR set?** → good, continue

```
(void)I2C1_SR1; (void)I2C1_SR2;
```

> **ADDR clear করার পদ্ধতি**: প্রথমে SR1 পড়ো, তারপর SR2 পড়ো → hardware ADDR auto-clear করে। এটা STM32 এর خاص requirement। ভুল করলে bus stuck হয়ে যায়।

ধাপ ৬ — Data byte পাঠানো

```
g = GUARD; while (!(I2C1_SR1 & (1 << 7)) && --g);
I2C1_DR = data;
```

> `SR1 bit7 = TXE (Transmit Empty)` → আগের byte পাঠানো শেষ, DR খালি। এবার আসল `data` ('S' বা 'K') write।

ধাপ ৭ — Data ACK wait + STOP

```
g = GUARD; while (!(I2C1_SR1 & (1 << 2)) && --g);
I2C1_CR1 |= (1 << 9);
return 1;
```

- `SR1 bit2 = BTF (Byte Transfer Finished)` → slave data টা ACK করেছে ও transfer শেষ
- `CR1 bit9 = STOP` → STOP condition (SCL high থাকতে SDA low→high)
- `return 1` → **success!**

5️⃣ `i2c_read()` — slave থেকে ১ byte পড়া

```
static int i2c_read(unsigned char addr, unsigned char *out) {
```

ধাপ ১ — ACK mode on + bus free

```
I2C1_CR1 |= (1 << 10);
```

`CR1 bit10 = ACK`। Master হিসেবে আমরা ACK দেব (default behavior)।

```
g = GUARD; while ((I2C1_SR2 & (1 << 1)) && --g); if (!g) return 0;
I2C1_CR1 |= (1 << 8);   // START
g = GUARD; while (!(I2C1_SR1 & (1 << 0)) && --g); ...
```

(একই ধাপ ২-৩, আগের মতো)

ধাপ ২ — Address + READ bit

```
I2C1_DR = (addr << 1) | 1;
```

`| 1` → LSB=1 → **READ** operation। Slave এখন data পাঠাবে।

```
g = GUARD; while (!(I2C1_SR1 & ((1<<1)|(1<<10))) && --g);
if (I2C1_SR1 & (1 << 10)) { ... return 0; }
if (!(I2C1_SR1 & (1 << 1))) { ... return 0; }
```

(ACK check, আগের মতো)

ধাপ ৩ — CRITICAL — NACK দিয়ে single byte পড়া

```
I2C1_CR1 &= ~(1 << 10);   /* NACK the single byte */
(void)I2C1_SR1; (void)I2C1_SR2;
I2C1_CR1 |= (1 << 9);     /* STOP */
```

- **কেন NACK?** আমরা মাত্র ১ byte চাই। Master যদি ACK দেয়, slave আরেক byte পাঠাবে। তাই শেষ byte এর আগেই ACK off (NACK) করতে হয় → "আর দিও না"।
- **Order important**: ACK clear করা, তারপর SR1/SR2 clear, তারপর STOP। এই সঠিক order না হলে hardware ভুলভাবে data পাঠাবে।
- STOP দিলে slave জানে "transfer শেষ"।

ধাপ ৪ — RXNE wait + data পড়া

```
g = GUARD; while (!(I2C1_SR1 & (1 << 6)) && --g);
if (!g) return 0;
*out = (unsigned char)I2C1_DR;
return 1;
```

- `SR1 bit6 = RXNE (Receive Not Empty)` → DR এ data এসেছে
- `I2C1_DR` পড়লে `*out` এ save হয়
- `return 1` → success

6️⃣ `main()` — Initialization

Clock enable

```
RCC_AHB1ENR |= (1 << 0) | (1 << 1) | (1 << 2);   /* GPIOA + GPIOB + GPIOC */
RCC_APB1ENR |= (1 << 21);                        /* I2C1 */
```

- `AHB1ENR bit0/1/2` → GPIOA/B/C clock on
- `APB1ENR bit21` → I2C1 clock on
- **Clock না দিলে register write কাজ করবে না** → প্রথম কাজ।

PA5 = LED output

```
GPIOA_MODER &= ~(3 << 10);  GPIOA_MODER |= (1 << 10);
```

- `MODER` ২ bit per pin। PA5 → bits 11:10।
- `~ (3<<10)` = bits 11:10 clear
- `(1<<10)` = value `01` = **General Purpose Output**
- এখন `GPIOA_ODR bit5` = 1 → LED on, 0 → off

PB8/PB9 = I2C AF, open-drain, pull-up

```
GPIOB_MODER  &= ~((3 << 16) | (3 << 18));
GPIOB_MODER  |=  ((2 << 16) | (2 << 18));
```

- PB8 → bits 17:16, PB9 → bits 19:18
- `2` = `10` = **Alternate Function mode**

```
GPIOB_OTYPER |=  (1 << 8) | (1 << 9);
```

> `OTYPER` ১ bit per pin। `1` = **open-drain**। **I2C এ বাধ্যতামূলক** (কারণ bus এ একাধিক device, wire-AND দরকার)।

```
GPIOB_PUPDR  &= ~((3 << 16) | (3 << 18));
GPIOB_PUPDR  |=  ((1 << 16) | (1 << 18));
```

`01` = **pull-up**। (যদিও external 4.7kΩ pull-up আছে, internal কেউ ব্যবহার করে — inert হিসেবে ক্ষতি নেই)।

```
GPIOB_AFRH   &= ~((0xF << 0) | (0xF << 4));
GPIOB_AFRH   |=  ((4 << 0) | (4 << 4));
```

- `AFRH` (AF Register High) → pins 8-15 এর জন্য। PB8 → bits 3:0, PB9 → bits 7:4।
- `4` = **AF4** = I2C1 alternate function (Datasheet এর AF table দেখো)
- AFRH এর nibble index: PB8 = nibble 0, PB9 = nibble 1

PC13 = button input pull-up

```
GPIOC_PUPDR  &= ~(3 << 26);  GPIOC_PUPDR |= (1 << 26);
```

- PC13 → bits 27:26
- `01` = **pull-up** → button না চাপলে HIGH, চাপলে LOW

I2C1 configuration

```
I2C1_CR1 |= (1 << 15);  I2C1_CR1 &= ~(1 << 15);
```

`CR1 bit15 = SWRST` → software reset। **I2C enable করার আগে reset করা ভালো practice** — bus stuck থাকলে clear হয়ে যায়।

```
I2C1_CR2 = 16;
```

`CR2 bits 5:0 = FREQ` → APB1 clock = 16 MHz (F446 default)। **Timing calc এ দরকার।**

```
I2C1_CCR = 80;
```

`CCR` = Clock Control Register। Standard mode এ `CCR = PCLK1 / (2 × SCL) = 16,000,000 / (2 × 100,000) = 80`। → **SCL = 100 kHz** (standard mode)

```
I2C1_TRISE = 17;
```

`TRISE` = maximum rise time। Standard mode এ: `TRISE = FREQ + 1 = 16 + 1 = 17`। datasheet অনুযায়ী।

```
I2C1_CR1 |= (1 << 0);
```

`CR1 bit0 = PE (Peripheral Enable)`। এবার I2C1 কাজ করছে।

7️⃣ Main loop — State Machine

```
int prev = 0, prevB = 0;
```

- `prev` = master button এর আগের state
- `prevB` = slave button এর আগের state

- **কেন?** শুধু rising edge (release→press) ধরতে। না হলে button চেপে ধরে থাকলে বারবার trigger হবে।

Part A — MY button → tell slave

```
int now = pressed();
if (now && !prev) {
    delay_ms(20);
    if (pressed() && i2c_write(SLAVE_ADDR, 'S')) {
        delay_ms(SLAVE_BLINK_WAIT);
        little_blink();
    }
}
prev = now;
```

1. `now = pressed()` → এখন চাপা?
2. `now && !prev` → **rising edge** — এইমাত্র চাপলো
3. `delay_ms(20)` → **debounce** (mechanical bounce এড়াতে)
4. আবার `pressed()` → এখনো চাপা? (real press confirm)
5. `i2c_write(0x28, 'S')` → slave এর কাছে `'S'` পাঠাও
6. Success? → 2.5 sec অপেক্ষা করো (slave এর ৫বার blink শেষ হওয়ার জন্য)
7. তারপর `little_blink()` → "আমার দিক থেকে ACK"

Part B — Poll slave's button

```
unsigned char r = 0;
if (i2c_read(SLAVE_ADDR, &r)) {
    int nowB = (r == 'B');
    if (nowB && !prevB) {
        blink(5);
        i2c_write(SLAVE_ADDR, 'K');
    }
    prevB = nowB;
}
```

1. `i2c_read` → slave এর button state নাও
2. `r == 'B'` মানে slave pressed
3. `nowB && !prevB` → slave button এর **rising edge**
4. `blink(5)` → আমার LED ৫বার (reaction)
5. `i2c_write(0x28, 'K')` → slave কে বলো "তোমার press আমি দেখেছি" → slave little-blink করবে
6. `prevB = nowB` → state মনে রাখো

```
delay_ms(10);
```

Polling rate ১০ms — bus এ চাপ কম রাখার জন্য।

🟢 PART 2: SLAVE Code — Line by Line

Slave কখনো নিজে bus শুরু করে না — **সবসময় master এর address match এর অপেক্ষায় থাকে।**

1️⃣ Slave-specific initialization

Address register — সবচেয়ে important

```
I2C1_OAR1  = (MY_ADDR << 1) | (1 << 14);
```

- `OAR1` = Own Address Register 1
- `(0x28 << 1)` = `0x50` → 7-bit address এর উপরের 7 bit
- `(1 << 14)` = **bit14 must be 1** for 7-bit mode (10-bit mode এ 0)। Datasheet এ clear বলা আছে।
- এই address এর সাথে master এর পাঠানো address মিললেই slave respond করবে।

ACK + Enable

```
I2C1_CR1  |= (1 << 10) | (1 << 0);
```

- `bit10 = ACK` → slave হিসেবে আমরা ACK দেব
- `bit0 = PE` → peripheral enable

2️⃣ Main loop — Event-driven

```
while (1) {
    while (!(I2C1_SR1 & (1 << 1)));           /* ADDR: addressed by master */
```

- `SR1 bit1 = ADDR` set হয় যখন **master এই slave এর address পাঠায়**।
- এখানে infinite wait। Master সত্যিই drives bus, তাই থাকবো।
- একবার ADDR set → master আমাদের ডেকেছে, ব্যাপার শুরু।

```
unsigned int sr2 = I2C1_SR2;              /* clears ADDR, gives direction */
```

একটা register read এ দুই কাজ:

1. SR2 পড়লে hardware **ADDR auto-clear** করে (SR1 তারপর SR2 পড়া STM32 এর requirement)
2. `SR2 bit2 = TRA (Transmitter/Receiver)` → আমরা এখন কে?
   - `TRA=0` → master **আমাকে writing** (আমি receiver)
   - `TRA=1` → master **আমাকে reading** (আমি transmitter)

Case 1: Master is WRITING to me (TRA=0)

```
if (!(sr2 & (1 << 2))) {
    while (!(I2C1_SR1 & (1 << 6)));         /* RXNE: data ready */
```

```
    unsigned char cmd = (unsigned char)I2C1_DR;
```

- `SR1 bit6 = RXNE` → data register এ byte এসেছে
- `I2C1_DR` পড়লে byte টা `cmd` এ save হয়

```
    while (!(I2C1_SR1 & (1 << 4)));         /* STOPF: master sent STOP */
    (void)I2C1_SR1; I2C1_CR1 |= (1 << 0);
```

- `SR1 bit4 = STOPF` → master STOP দিয়েছে
- `(void)I2C1_SR1` পড়লে STOPF clear হয় (read-only bit)
- `CR1 |= (1<<0)` → PE re-enable (safety)। কিছু STM32 এ STOP এর পর reset দরকার।

```
    if      (cmd == 'S') blink(5);          /* master pressed → I blink */
    else if (cmd == 'K') little_blink();    /* my press acked           */
}
```

- `'S'` = master আমার সাথে কথা বলতে চায় → আমি ৫বার blink
- `'K'` = master আমার press acknowledge করেছে → আমি ছোট ২বার blink

Case 2: Master is READING from me (TRA=1)

```
else {
    while (!(I2C1_SR1 & (1 << 7)));         /* TXE: transmit empty */
    I2C1_DR = pressed() ? 'B' : 0x00;       /* send button state */
```

- `SR1 bit7 = TXE` → DR খালি, byte লিখতে পারি
- **আমার button state পাঠাই**: চাপা থাকলে `'B'`, নাহলে `0x00`
- Master `i2c_read` এ এই value পাবে।

```
    while (!(I2C1_SR1 & (1 << 10)));        /* AF: master NACKed after 1 byte */
    I2C1_SR1 &= ~(1 << 10);
}
```

- `SR1 bit10 = AF (Acknowledge Failure)` → master এক byte পড়ে **NACK** দিয়েছে → transfer শেষ
- AF bit clear করে পরের transfer এর জন্য প্রস্তুত

🔄 Full Workflow — একটা উদাহরণ দিয়ে

Example 1: Master এর button চাপা

```
T=0ms:    [MASTER] pressed()=1 (rising edge detected)
          i2c_write(0x28, 'S'):
          ┌─ START
          ├─ Address 0x50 (0x28<<1 | 0) → [SLAVE] ADDR set
          ├─ Data 'S'
          └─ STOP
```

```
T≈1ms:    [SLAVE] ADDR flag → SR2 read (TRA=0, writing)
          RXNE wait → cmd='S'
          STOPF wait → clear
          cmd=='S' → blink(5) শুরু (2000ms)

T=1ms-2501ms: [SLAVE] LED: on/off ×5
              [MASTER] delay_ms(2500) — অপেক্ষা

T=2501ms: [MASTER] little_blink() — 120ms ×2 (240ms)
          [MASTER] loop → prev = 1

T=2511ms: [MASTER] read slave → 0x00 (slave button চাপা না)
          prevB = 0
```

Example 2: Slave এর button চাপা

```
T=0ms:    [SLAVE] user button চাপে (local only, bus event না)
          [SLAVE] loop: ADDR flag not set → waiting

T=10ms:   [MASTER] polling: i2c_read(0x28, &r):
          ┌─ START
          ├─ Address 0x51 (0x28<<1 | 1) → READ → [SLAVE] ADDR set
          ├─ [SLAVE] SR2 read → TRA=1
          ├─ [SLAVE] DR = 'B'
          ├─ [MASTER] NACK after 1 byte
          └─ STOP

T≈10ms:   [MASTER] r = 'B'
          nowB=1, prevB=0 → rising edge!
          blink(5) → 2000ms
          i2c_write(0x28, 'K') → [SLAVE] ADDR, TRA=0, cmd='K'
                                 [SLAVE] little_blink() → 240ms

T≈2010ms: [MASTER] loop, prevB=1
```

Example 3: Slave unplugged (safety test)

```
[MASTER] i2c_read(0x28):
  START
  Address 0x51 → [no slave]
  GUARD counter count down...
  AF flag set at some point
  → return 0

[MASTER] i2c_read fails → skip
          Main loop continues — NO HANG ✅
```

🔑 Exam এ মনে রাখার Key Points

HALF-DUPLEX কেন?

- I2C এ SDA একটাই wire → **একসাথে দুই দিকে data যায় না**
- Master একটা transaction এ হয় write, নয় read
- কিন্তু ক্রমানুসারে write + read করা যায় → "two-way"
- **Full duplex** হলে আলাদা TX/RX wire লাগবে (UART, SPI) — I2C তে নেই

I2C এ Master vs Slave এর ভূমিকা

	Master	Slave
Clock (SCL)	generate করে	follow করে
START/STOP	দেয়	wait করে
Address	পাঠায়	match করে
Bus control	সব	কখনো না
ACK	read এ দেয়	প্রতিটা byte এ দেয়

গুরুত্বপূর্ণ Register bits (exam এ আসবে)

Register	Bit	নাম	মানে
CR1	0	PE	Peripheral Enable
CR1	8	START	START condition
CR1	9	STOP	STOP condition
CR1	10	ACK	ACK enable
CR1	15	SWRST	Software reset
SR1	0	SB	START bit sent
SR1	1	ADDR	Address matched
SR1	2	BTF	Byte transfer finished
SR1	6	RXNE	RX data register not empty

Register	Bit	নাম	মানে
SR1	7	TXE	TX data register empty
SR1	10	AF	Acknowledge failure
SR2	1	BUSY	Bus busy
SR2	2	TRA	Transmitter/Receiver (slave)

Timing Calculation (exam এ numerical)

```
CCR  = PCLK1 / (2 × SCL) = 16MHz / (2×100kHz) = 80
TRISE = FREQ + 1 = 16 + 1 = 17
```

Critical "কেন এভাবে?" প্রশ্ন

Q: STOP এর আগে ACK clear করে কেন read এ? A: আমরা ১ byte চাই। ACK রাখলে slave আরেক byte পাঠাবে। NACK দিয়ে "আর দিও না" বলতে হয়।

Q: SR1 তারপর SR2 পড়তে হয় কেন? A: STM32 hardware এর sequence। একসাথে না পড়লে ADDR flag clear হয় না → bus stuck।

Q: Slave এ `while` loop এর ভেতর আরেক `while` কেন (infinite wait)? A: Slave passive — bus কখনো hang হবে না কারণ master সত্যিই drive করে। Master এর মতো GUARD লাগে না (তবে robust করতে চাইলে GUARD দিতে পারো)।

Q: `AF` bit slave এ কেন? A: Master এক byte পড়ে NACK দেয় → hardware AF set করে → slave জানে "transfer শেষ"।

Q: `OAR1 bit14 = 1` কেন? A: STM32F446 এ 7-bit addressing mode এ bit14 অবশ্যই 1 হতে হবে (Reference Manual)। না হলে address কাজ করবে না।

📝 Quick Cheat Sheet — Exam এর আগে

```
MASTER init order:
  1. RCC enable (GPIO + I2C clock)
  2. GPIO: PA5 output, PB8/9 = AF4 + open-drain + pull-up
  3. I2C: SWRST → FREQ=16 → CCR=80 → TRISE=17 → PE=1
  4. Main loop: debounce button → i2c_write; poll → i2c_read

SLAVE init order:
  1. RCC enable
  2. GPIO: same as master
  3. I2C: SWRST → OAR1=(addr<<1)|(1<<14) → FREQ=16 → CCR=80 → TRISE=17 → ACK+PE
```

```
  4. Main loop: wait ADDR → SR2 read → if TRA=0 (write) handle cmd; else (read) send button

I2C write sequence:
  BUSY wait → START → SB wait → DR=addr<<1|0 → ADDR wait → SR1/SR2 read
  → TXE wait → DR=data → BTF wait → STOP → return 1

I2C read sequence:
  ACK on → BUSY wait → START → SB wait → DR=addr<<1|1 → ADDR wait
  → ACK off (NACK) → SR1/SR2 read → STOP → RXNE wait → read DR
```

শুভকামনা তোমার lab exam এর জন্য! 🎓 যদি কোনো নির্দিষ্ট অংশ আরও গভীরভাবে বুঝতে চাও (যেমন I2C timing diagram, বা ADDR clear এর exact sequence), বলো — বিস্তারিত দেখাবো।

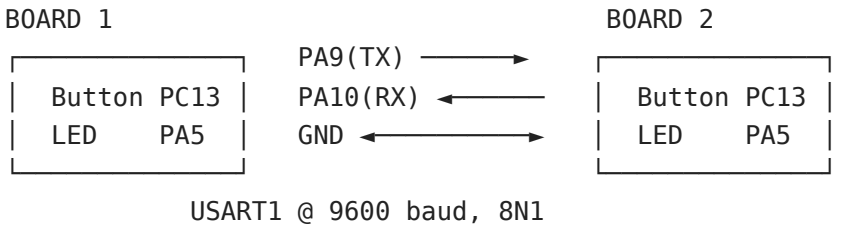
Pasted text.txt Pasted text.txt

UART code

UART Two-Way Switch — Full Explanation (NUCLEO-F446RE)

তোমার I2C এর মতোই এই code, কিন্তু **UART** দিয়ে। সবচেয়ে মজার ব্যাপার — **দুই board এ একই code চলে** (Master == Slave)। কারণ UART এ master/slave concept নেই, দুই দিকেই সমান।

🎯 Big Picture — I2C vs UART



Feature	I2C	UART
Wires	2 (SCL+SDA)	2 (TX+RX) + GND
Clock	SCL shared (synchronous)	**none** (asynchronous)
Master/Slave	আছে	**নেই**
Duplex	half-duplex (এক wire)	**full-duplex** (TX/RX আলাদা)
Addressing	address দিয়ে	নেই (point-to-point)

Feature	I2C	UART
একই code দুই board এ?	❌ না (master ≠ slave code)	✅ হ্যাঁ

কেন একই code? — UART এ address/bus control concept নেই। দুই board ই একইভাবে "send করো, receive করো"। তাই একটা code দুই জায়গায় flash করলেই হয়।

📋 Scenario Table (কোনটা কী trigger করে)

#	কে trigger করলো	Pressed board কী করে	Other board কী করে
1	**Board A** button	`'S'` send করে, `awaiting=1`	`'S'` পায় → `blink(5)` → `'D'` send করে
2	**Board B**	`'D'` পায়, `awaiting` set ছিল	—
3	Result	`little_blink()` (2×120ms)	LED ৫বার blink হয়ে বন্ধ
4	কেউ চাপে না	—	— (দুটোই dark)
5	Reply না আসে (other board off)	6 sec পর `awaiting=0`, hang করে না ✅	—

🔵 Line by Line — Register Definitions

```
#define RCC_AHB1ENR  (*(volatile unsigned int *)0x40023830)
#define RCC_APB2ENR  (*(volatile unsigned int *)0x40023844)
#define GPIOA_MODER  (*(volatile unsigned int *)0x40020000)
#define GPIOA_PUPDR  (*(volatile unsigned int *)0x4002000C)
#define GPIOA_ODR    (*(volatile unsigned int *)0x40020014)
#define GPIOA_AFRH   (*(volatile unsigned int *)0x40020024)
#define GPIOC_PUPDR  (*(volatile unsigned int *)0x4002080C)
#define GPIOC_IDR    (*(volatile unsigned int *)0x40020810)
#define USART1_SR    (*(volatile unsigned int *)0x40011000)
#define USART1_DR    (*(volatile unsigned int *)0x40011004)
#define USART1_BRR   (*(volatile unsigned int *)0x40011008)
#define USART1_CR1   (*(volatile unsigned int *)0x4001100C)
```

STM32F446 reference manual থেকে:

- **RCC_APB2ENR** → USART1 clock enable (bit4)
- **GPIOA_AFRH** → PA9/PA10 alternate function (USART1 = AF7)
- **USART1_SR** → Status Register
- **USART1_DR** → Data Register (read = receive, write = transmit)

- **USART1_BRR** → Baud Rate Register
- **USART1_CR1** → Control Register 1

`volatile` = compiler কে বলছে "এই address এর value hardware বদলাতে পারে, প্রতিবার actual memory থেকে পড়ো, cache/optimize কোরো না"।

🔧 Helper Functions

1️⃣ `delay_ms` — Software delay

```
static void delay_ms(unsigned int ms) {
    for (unsigned int m = 0; m < ms; m++)
        for (volatile unsigned int i = 0; i < 1300; i++);
}
```

- Nested loop, ১৩০০ iterations ≈ ১ms (১৬ MHz এ approx)
- Hardware timer নয়, শুধু busy-wait। Lab এ চলে, কিন্তু accurate না।

2️⃣ `pressed` — Button read

```
static int pressed(void) { return (GPIOC_IDR & (1 << 13)) == 0; }
```

- PC13 এর input data read
- **Active LOW** — pull-up আছে, চাপলে LOW → `==0` = pressed
- চাপা না থাকলে HIGH → `!=0` = 0 (false)

3️⃣ `blink(5)` — "Other board pressed"

```
static void blink(int times) {
    for (int i = 0; i < times; i++) {
        GPIOA_ODR |=  (1 << 5);  delay_ms(BLINK_MS);   // LED on 200ms
        GPIOA_ODR &= ~(1 << 5);  delay_ms(BLINK_MS);   // LED off 200ms
    }
}
```

- `BLINK_MS = 200` → মোট `5 × (200+200) = 2000ms`
- `|= (1<<5)` → PA5 high (LED on)
- `&= ~(1<<5)` → PA5 low (LED off)
- **Purpose**: other board press করেছে → আমার LED বলছে "পেয়েছি"

4️⃣ `little_blink` — "My press acked"

```
static void little_blink(void) {
    for (int i = 0; i < 2; i++) {
        GPIOA_ODR |=  (1 << 5);  delay_ms(120);
```

```
        GPIOA_ODR &= ~(1 << 5);  delay_ms(120);
    }
}
```

- ২বার × 120ms (মোট ~480ms)
- **Purpose**: other board এর ACK (`'D'`) এসেছে → আমি confirm দিচ্ছি

5️⃣ `send` — UART transmit

```
static void send(char c) {
    while (!(USART1_SR & (1 << 7)));   // wait TXE
    USART1_DR = c;
}
```

- **`USART1_SR bit7 = TXE`** (Transmit data register Empty)
  - `1` = DR খালি, byte লিখতে পারি
  - `0` = আগের byte এখনো পাঠানো হয়নি
- `while(!(...))` → TXE=1 হওয়া পর্যন্ত অপেক্ষা
- `USART1_DR = c` → hardware byte টা TX pin দিয়ে পাঠায়
- **কেন `while` দরকার?** UART slow (9600 baud ~1ms/byte)। DR এ overwrite করলে আগের byte হারিয়ে যাবে (overrun)। তাই buffer খালি কিনা দেখে।

🚀 `main()` — Initialization (Line by Line)

Step 1 — Clock enable

```
RCC_AHB1ENR |= (1 << 0) | (1 << 2);   /* GPIOA + GPIOC */
RCC_APB2ENR |= (1 << 4);              /* USART1 */
```

- **AHB1ENR bit0** = GPIOA clock
- **AHB1ENR bit2** = GPIOC clock (button PC13 এর জন্য)
- **APB2ENR bit4** = USART1 clock
- **Clock না দিলে register write silently fail** → প্রথমেই clock enable করতে হয়।

Step 2 — PA5 = LED output

```
GPIOA_MODER &= ~(3 << 10);  GPIOA_MODER |= (1 << 10);
```

- `MODER` = ২ bit per pin
- PA5 → bits 11:10
- `~(3<<10)` = bits 11:10 clear
- `(1<<10)` = value `01` = **General Purpose Output**
- এখন PA5 এর ODR bit5 = LED on/off

Step 3 — PA9 (TX) / PA10 (RX) = AF7 (USART1)

```
GPIOA_MODER &= ~((3 << 18) | (3 << 20));
GPIOA_MODER |=  ((2 << 18) | (2 << 20));
```

- PA9 → bits 19:18
- PA10 → bits 21:20
- `2` = `10` = **Alternate Function mode**

```
GPIOA_AFRH  &= ~((0xF << 4) | (0xF << 8));
GPIOA_AFRH  |=  ((7 << 4) | (7 << 8));
```

- **AFRH** = Alternate Function Register High (pins 8–15)
- PA9 → AFRH bits 7:4 (nibble 1)
- PA10 → AFRH bits 11:8 (nibble 2)
- **7 = AF7** = USART1 (Datasheet AF table)
- **কেন AF7?** — STM32F446 datasheet: PA9 = USART1_TX (AF7), PA10 = USART1_RX (AF7)। ভুল AF দিলে UART কাজ করবে না।

Step 4 — RX pull-up

```
GPIOA_PUPDR &= ~(3 << 20);  GPIOA_PUPDR |= (1 << 20);      /* RX pull-up */
```

- PA10 → bits 21:20
- `01` = **pull-up** enabled
- **কেন RX pull-up?** — UART idle state = HIGH। TX disconnected হলে RX floating থাকলে garbage read হতে পারে। Pull-up দিয়ে idle HIGH নিশ্চিত করা হয়।

Step 5 — PC13 pull-up (button)

```
GPIOC_PUPDR &= ~(3 << 26);  GPIOC_PUPDR |= (1 << 26);      /* PC13 pull-up */
```

- PC13 → bits 27:26
- `01` = pull-up → button না চাপলে HIGH, চাপলে LOW

Step 6 — Baud Rate Register

```
USART1_BRR = 0x0683;                          /* 9600 @ 16MHz */
```

Calculation: USART clock = PCLK2 = 16 MHz (F446 default)।

```
USARTDIV = fCK / (16 × baud) = 16,000,000 / (16 × 9600) = 104.1666...
BRR mantissa = 104 = 0x68
BRR fraction = 0.1666 × 16 = 2.67 ≈ 3 = 0x3
BRR = 0x0683
```

Verify: `0x683 = 104×16 + 3 = 1667`। `16000000 / 1667 = 9598 bps ≈ 9600` ✅

- BRR এ integer+fraction একসাথে pack করা হয় (12-bit mantissa + 4-bit fraction)
- ভুল **BRR = garbage data** (differing baud = framing error)

Step 7 — USART Control Register

```
USART1_CR1 = (1 << 13) | (1 << 3) | (1 << 2);
```

Bit	Name	Value	Meaning
13	UE	1	USART Enable
3	TE	1	Transmitter Enable
2	RE	1	Receiver Enable

- **UE (bit13)** = overall USART on
- **TE (bit3)** = TX pin active (send করা যাবে)
- **RE (bit2)** = RX pin active (receive করা যাবে)
- **Word length** default = 8-bit (bit12 M=0), **parity** default off → **8N1** format

---

🔄 Main Loop — State Machine

```
int prev = 0, awaiting = 0;
unsigned int timer = 0;
```

- **`prev`** = button এর আগের state (rising edge detect এর জন্য)
- **`awaiting`** = আমি `'S'` পাঠিয়েছি, `'D'` ACK এর অপেক্ষায়
- **`timer`** = ACK timeout countdown (ms)

Part A — MY button pressed

```
int now = pressed();
if (now && !prev) {
    delay_ms(20);                                  /* debounce */
    if (pressed()) { send('S'); awaiting = 1; timer = ACK_TIMEOUT; }
}
prev = now;
```

Line by line:

1. **`int now = pressed()`** → এখন চাপা?
2. **`now && !prev`** → **rising edge** (আগে চাপা ছিল না, এখন চাপা) — একবারই trigger
3. **`delay_ms(20)`** → **debounce** (mechanical switch bounce এড়াতে ২০ms অপেক্ষা)
4. **`if (pressed())`** → debounce এর পরেও চাপা আছে? (real press confirm)
5. **`send('S')`** → other board কে বলো "আমি চাপ দিয়েছি, তুমি blink করো"

6. **`awaiting = 1`** → "আমি এখন ACK এর অপেক্ষায় আছি"
7. **`timer = ACK_TIMEOUT`** → 6000ms timeout start
8. **`prev = now`** → state save

কেন `awaiting` flag?

- আমি `'S'` পাঠিয়েছি, other board blink করে `'D'` পাঠাবে
- `'D'` আসলে `little_blink()` করব
- কিন্তু `'D'` যদি না আসে (other board off)? → timeout এড়াতে `timer`

Part B — Receive from other board

```
if (USART1_SR & (1 << 5)) {
    char c = (char)USART1_DR;
    if (c == 'S') {
        blink(5);
        send('D');
    } else if (c == 'D' && awaiting) {
        little_blink();
        awaiting = 0;
    }
}
```

Line by line:

1. **`USART1_SR & (1 << 5)`** → **`SR bit5 = RXNE`** (Receive Not Empty)
   - RXNE=1 → DR এ একটা byte এসে গেছে
   - Polling করে check (interrupt নয়)
2. **`char c = (char)USART1_DR`** → data read। **DR পড়লে RXNE auto-clear** হয়।
3. **`if (c == 'S')`** → other board বলছে "আমি চাপ দিয়েছি"
   - **`blink(5)`** → আমার LED ৫বার (2000ms)
   - **`send('D')`** → ACK ফেরত পাঠাও "দেখেছি"
4. **`else if (c == 'D' && awaiting)`** → other board আমার `'S'` এর ACK পাঠিয়েছে
   - **`awaiting` check জরুরি** — যদি আমি কিছু না পাঠিয়ে থাকি, stray `'D'` ignore করো
   - **`little_blink()`** → ২বার 120ms — "ACK পেয়েছি"
   - **`awaiting = 0`** → আর অপেক্ষা করব না

Critical point: `'S'` আর `'D'` handling — একই code দুই দিকে। কারণ source জানা যায় না, শুধু message চেনা যায়।

Part C — Timeout

```
if (awaiting && --timer == 0) awaiting = 0;
```

- **`awaiting`** = 1 হলে decrement
- `--timer == 0` → 6000 → 0 হয়ে গেছে (6 sec পার)

- **`awaiting = 0`** → give up। other board হয়তো off, ACK আসবে না
- **কেন দরকার?** — `awaiting` থাকলে পরের press ঠিকমতো handle হবে না (state stuck)। এটা software timeout।

Note: Actually এই decrement rate `delay_ms(1)` প্রতি loop ≈ 1ms। কিন্তু blink(5) 2000ms চলে, তাই loop মাঝে মাঝে দেরি করে। এটা ideal না, কিন্তু lab এ চলে।

Part D — Loop delay

```
delay_ms(1);
```

- প্রতি loop এ 1ms delay
- **কেন?** CPU busy-wait spam কমাতে, আর `timer` count কে roughly 1ms এর সাথে synchronize করতে
- polling rate ≈ 1ms। 9600 baud এ byte আসতে ~1ms লাগে → ঠিক সময়ে ধরা পড়ে।

🔁 Full Workflow — Example

Example 1: Board A এর button চাপা

```
T=0ms      [Board A] prev=0
           pressed()=1 → rising edge
           delay_ms(20) → still pressed
           send('S') → UART TX
           awaiting=1, timer=6000

T≈1ms      [Board B] RXNE=1
           c='S'
           → blink(5) শুরু → 2000ms
           → send('D')

T=1..2001ms  [Board B] LED on/off ×5

T≈2002ms   [Board A] RXNE=1
           c='D', awaiting=1 → OK
           → little_blink() → ~480ms
           → awaiting=0

T≈2482ms   দুটোই idle। পরের press এর জন্য ready।
```

Example 2: Board B এর button চাপা (Board A একই behavior)

Same as above, শুধু A আর B swap।

Example 3: Other board off/unplugged

```
T=0ms      [Board A] send('S'), awaiting=1, timer=6000
T=1..6000ms  কোন reply নেই
T=6000ms   --timer==0 → awaiting=0
           পরের press কাজ করবে ✅ (hang নেই)
```

Example 4: Rapid double press

```
[Board A] press → awaiting=1
[Board A] আবার press → rising edge → send('S') আবার → awaiting=1, timer reset
```

- এই code এ দ্বিতীয় `'S'` পাঠালে Board B আবার blink করবে। `awaiting` reset হয়।
- **Limitation**: `awaiting=1` থাকলেও নতুন press হলে ignore হয় না। Production এ এটা "already awaiting" check দিয়ে ঠিক করতে হয়।

📊 UART Status Register — যা যা লাগে

USART1_SR key bits

Bit	Name	Meaning	ব্যবহার
7	**TXE**	TX Empty	`send()` এ wait
6	**TC**	Transmission Complete	(এখানে ব্যবহার হয়নি)
5	**RXNE**	RX Not Empty	Receive check এ wait
4	IDLE	Idle line detected	(না)
3	ORE	Overrun Error	(না)
2	NF	Noise	(না)
1	FE	Framing Error	(না)
0	PE	Parity Error	(না)

Clear করার নিয়ম:

- **TXE** → DR এ write করলে auto-clear
- **RXNE** → DR read করলে auto-clear
- **TC** → DR read করলে auto-clear

USART1_CR1 key bits

Bit	Name	Purpose
13	**UE**	USART Enable
12	M	Word length (0=8bit)
10	PCE	Parity Control Enable
9	PS	Parity Select
5	RXNEIE	RX interrupt enable (এখানে off — polling)
3	**TE**	Transmit Enable
2	**RE**	Receive Enable

🔬 Timing Calculation (exam এ numerical)

Baud Rate (9600 @ 16MHz)

```
USARTDIV = PCLK2 / (16 × baud)
         = 16,000,000 / (16 × 9600)
         = 104.1666

Mantissa (integer)  = 104 = 0x68
Fraction × 16       = 0.1666 × 16 = 2.67 ≈ 3 = 0x3
BRR                 = (104 << 4) | 3 = 0x0683
```

Check: `16,000,000 / 1667 = 9598 bps` — 9600 এর 0.02% error (acceptable, <2%) ✅

Byte transfer time

```
bits per byte = 1 start + 8 data + 0 parity + 1 stop = 10 bits
time = 10 / 9600 ≈ 1.04 ms
```

তাই polling interval `delay_ms(1)` byte এর সাথে প্রায় match করে।

ACK_TIMEOUT 6000ms কেন?

- blink(5) = 2000ms
- Other board তখন reply `'D'` পাঠায়
- 6 sec = 3× এর বেশি safety margin
- এর মধ্যে reply না আসলে ধরে নাও other board dead

⚡ Critical Concepts — Exam এ আসবে

Q: UART এ Master/Slave নেই, তবে two boards এ একই code কেন চলে?

A: UART = asynchronous, point-to-point, full-duplex। Both boards একই ভাবে:

- নিজের button poll করে → send
- নিজের RX check করে → receive
- কোন board কে "master" বলার দরকার নেই। দুই দিকেই symmetric behavior। I2C এর মতো address/bus control concept নেই।

Q: `send()` এ `while(!TXE)` কেন জরুরি?

A: DR register এক byte ধরে। New byte লিখলে আগেরটা overwrite হয়। TXE=1 মানে "আগের byte পাঠানো শেষ, DR খালি"। না check করলে byte হারাবে (data corruption)।

Q: `awaiting` এবং `timer` কেন?

A: State machine এর heart:

- `awaiting=1` = "আমি `'S'` পাঠিয়েছি, `'D'` এর অপেক্ষায়"
- `timer` = timeout counter (other board না থাকলে hang হবে না)
- `'D'` পেলে `awaiting=0` → state clear

Q: RX pull-up (PA10) কেন?

A: UART idle state HIGH। TX unplugged/floating হলে RX random noise read করতে পারে → garbage। Pull-up দিয়ে idle HIGH force করা হয়।

Q: AF7 কেন PA9/PA10 এর জন্য?

A: STM32F446 datasheet এ Alternate Function table আছে। PA9 = USART1_TX (AF7), PA10 = USART1_RX (AF7)। অন্য AF দিলে pin USART1 এর সাথে connect হবে না।

Q: `blink(5)` এবং `little_blink()` এর পার্থক্য?

A:

- `blink(5)` = "**তুমি** press করেছ, **আমি** দেখালাম" — receiving board এর reaction
- `little_blink()` = "**আমার** press এর ACK পেলাম" — sending board এর confirmation
- দুটো আলাদা state visualize করে

Q: Full-duplex থাকলে একসাথে send+receive possible কেন এখানে ব্যবহার হল না?

A: UART hardware full-duplex capable, কিন্তু **code polling-based**। একটা loop এ:

1. button check
2. TX check
3. RX check

এই sequential structure এ send এর সময় blink(5) এর মধ্যে receive ব্লকড। True full-duplex এর জন্য **interrupt** বা **DMA** দরকার। এই code simple রাখতে polling use করেছে।

Q: 5-blink এর সময় board কি receive পারে?

A: না! `blink(5)` = 2000ms block। এই সময়ে RX data overflow হতে পারে (overrun error)। Production এ interrupt-based RX লাগে। Lab এর জন্য OK কারণ sender board শুধু একবার `'S'` পাঠায়।

📝 Quick Cheat Sheet — Exam এর আগে

```
UART INIT ORDER:
1. RCC enable (GPIOA + GPIOC + USART1)
2. PA5 = output (LED)
3. PA9/PA10 = AF mode + AFRH=7 (AF7)
4. PA10 = pull-up (RX idle HIGH)
5. PC13 = pull-up (button)
6. BRR = 0x0683 (9600 @ 16MHz)
7. CR1 = UE | TE | RE

MAIN LOOP:
- button rising edge → debounce → send('S') → awaiting=1, timer=6000
- RXNE? → read DR:
    'S' → blink(5) + send('D')
    'D' + awaiting → little_blink + awaiting=0
- awaiting && --timer==0 → awaiting=0 (timeout)

SEND:
  while(!TXE); DR = c;

RECEIVE:
  if(RXNE) { c = DR; ... }

BRR calc:
  USARTDIV = PCLK2 / (16 × baud)
  BRR = (mantissa << 4) | fraction

Key SR bits:
```

```
  bit7 TXE   (send wait)
  bit5 RXNE  (receive check)

Key CR1 bits:
  bit13 UE, bit3 TE, bit2 RE
```

🆚 I2C vs UART — একনজরে তুলনা (Exam এর জন্য)

Aspect	I2C	UART
Code	Master ≠ Slave	**Same code**
Address	আছে (0x28)	নেই
Direction	Half-duplex	Full-duplex capable
Clock	SCL (shared)	Internal baud
Wires	SCL+SDA	TX+RX
Speed	Config via CCR	Config via BRR
Multi-device	হ্যাঁ (address)	না (2 only)
Timing	SCL driven	Asynchronous
Status bits	SB, ADDR, TXE, RXNE, AF	TXE, RXNE
Error handling	GUARD counter	awaiting timeout

শুভকামনা lab exam এর জন্য! 🎓 দুটো protocol একসাথে পড়লে পার্থক্যটা মাথায় গেঁথে যাবে — I2C এ **master drives everything**, UART এ **symmetric peer-to-peer**। এই মূল concept টা exam এ লিখলে ভালো নম্বর পাবে।

কোনো নির্দিষ্ট অংশ (BRR calculation, interrupt-based version, বা timing diagram) আরও গভীরে লাগলে বলো — detail করে দেবো।

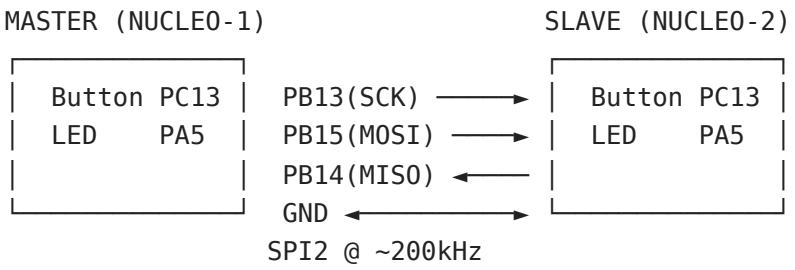
Pasted text.txt Pasted text.txt

SPI code

SPI Two-Way Switch — Full Explanation (NUCLEO-F446RE)

I2C আর UART এর পর এবার **SPI**। SPI এর নিজস্ব speciality আছে — এখানে **master আর slave এর code আলাদা**, কারণ SPI এ **master clock generate করে** আর **slave clock follow করে**। চলো step-by-step বুঝি।

🎯 Big Picture — SPI কীভাবে কাজ করে



SPI এর ৪টা wire:

Wire	Full Name	Direction	কাজ
SCK	Serial Clock	Master → Slave	Clock pulse (timing)
MOSI	Master Out Slave In	Master → Slave	Master এর data
MISO	Master In Slave Out	Slave → Master	Slave এর data
NSS	Slave Select	Master → Slave	Chip select (এখানে software)

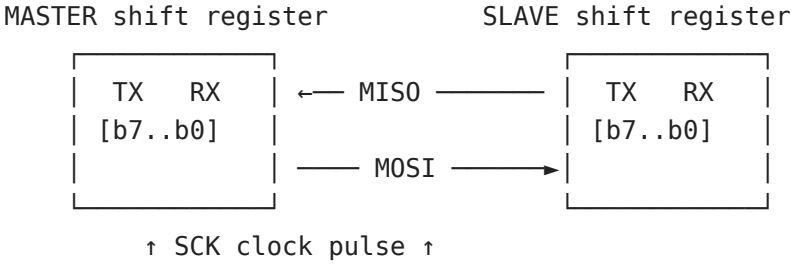
এখানে **PB13=SCK, PB14=MISO, PB15=MOSI** (NUCLEO-F446RE SPI2 pins)।

🆚 I2C / UART / SPI — তুলনা

Feature	I2C	UART	SPI
Wires	2 (SCL+SDA)	2 (TX+RX)	3 (SCK+MOSI+MISO)
Clock	Shared bus clock	None (async)	**Separate SCK**
Duplex	Half	Full	**Full** (simultaneous!)
Master concept	আছে	নেই	**আছে (master generates clock)**
Same code both?	❌	✅	❌ (master/slave আলাদা)

Feature	I2C	UART	SPI
Speed	100/400kHz	baud-based	**MHz range possible**
Address	আছে	নেই	নেই (NSS দিয়ে select)

🔑 SPI এর সবচেয়ে গুরুত্বপূর্ণ concept: Shift Register



প্রতিটা **SCK pulse** এ একইসাথে দুই দিকে **1 bit** যায়। 8 pulse এ 1 byte transfer।

- Master `SPI2_DR = x` write করলে যতক্ষণ না transfer শেষ, `SPI2_DR` read করলে **slave এর পাঠানো byte** পায়।
- এটাই SPI এর magic — **একটা transaction এ একইসাথে send আর receive হয়**।

🔵 PART 1: MASTER Code — Line by Line

1️⃣ Macros — Timing constants

```
#define BLINK_MS           200
#define SLAVE_BLINK_WAIT   2500  /* > slave's 5-blink time (2000 ms) */
#define SLAVE_LITTLE_WAIT  600   /* > slave's little-blink time (480 ms) */
```

- `BLINK_MS 200` → LED on/off 200ms
- `SLAVE_BLINK_WAIT 2500` → master `'S'` পাঠানোর পর 2.5sec অপেক্ষা (slave 5 blink ~2000ms + margin)
- `SLAVE_LITTLE_WAIT 600` → `'K'` পাঠানোর পর 600ms (slave little-blink ~480ms + margin)

কেন এই wait গুলো দরকার? SPI এ slave blink করা state এ থাকলে তার timing loop miss হয়ে MISO output ভুল আসতে পারে। তাই master কে "শান্ত থাকতে হয়" যতক্ষণ slave busy।

2️⃣ Registers (SPI2 specific)

```
#define SPI2_CR1     (*(volatile unsigned int *)0x40003800)
#define SPI2_SR      (*(volatile unsigned int *)0x40003808)
#define SPI2_DR      (*(volatile unsigned int *)0x4000380C)
```

- **SPI2_CR1** → Control Register 1 (mode, clock, enable)
- **SPI2_SR** → Status Register (TXE, RXNE, BSY)
- **SPI2_DR** → Data Register (TX/RX উভয়ই — read/write direction decide করে)

3️⃣ `spi_xfer()` — SPI এর heart 💓

```
static unsigned char spi_xfer(unsigned char b) {
    while (!(SPI2_SR & (1 << 1)));   // wait TXE (bit1) – TX buffer empty
    SPI2_DR = b;                     // load byte to send
    while (!(SPI2_SR & (1 << 0)));   // wait RXNE (bit0) – RX buffer not empty
    return (unsigned char)SPI2_DR;   // read received byte
}
```

একদম line-by-line:

Line	কী করছে	কেন
`while(!(SR & (1<<1)))`	**TXE=1** হওয়া পর্যন্ত wait	TX buffer খালি কিনা check। Full হলে byte overwrite হবে।
`SPI2_DR = b`	DR এ byte write	Hardware এটা shift register এ load করে, SCK pulse generate হয়, MOSI দিয়ে bit-by-bit যায়
`while(!(SR & (1<<0)))`	**RXNE=1** হওয়া পর্যন্ত wait	8 clock শেষে slave এর byte RX register এ এসেছে কিনা
`return SPI2_DR`	DR read	Slave এর পাঠানো byte পাওয়া গেল

🔑 গোপন কথা (exam এ জিজ্ঞেস করবে)

"একই DR register এ write করলাম, আবার read করলাম — এটা কীভাবে সম্ভব?"

- SPI এ DR register **দুই buffer** — TX buffer (write করে) আর RX buffer (read করে)
- একই address এ **write মানে TX**, **read মানে RX**
- `spi_xfer()` = "simultaneously send + receive" — এটাই SPI এর full-duplex nature

"কেন `spi_xfer(0x00)` দিয়ে dummy send করে data পড়া হয়?"

- SPI এ data "পেতে" হলে master কে clock generate করতে হয়
- Clock generate করতে হলে **কিছু একটা পাঠাতে হবে** (dummy `0x00`)
- এই dummy টা slave ignore করে, শুধু clock pulse দরকার
- **Receiving = Sending dummy in SPI** ← এটা exam এর classic question

---

4️⃣ `main()` — Initialization

Clock enable

```
RCC_AHB1ENR |= (1 << 0) | (1 << 1) | (1 << 2);   /* GPIOA + GPIOB + GPIOC */
RCC_APB1ENR |= (1 << 14);                        /* SPI2 */
```

- **SPI2 APB1 bus** এ (I2C এর সাথে same bus), তাই `RCC_APB1ENR`
- bit14 = SPI2 clock enable
- **Clock না দিলে register write ignore** → প্রথম কাজ

PA5 = LED output (আগের মতোই)

```
GPIOA_MODER &= ~(3 << 10);  GPIOA_MODER |= (1 << 10);          /* PA5 LED */
```

PB13/14/15 = AF5 (SPI2)

```
GPIOB_MODER &= ~((3 << 26) | (3 << 28) | (3U << 30));
GPIOB_MODER |=  ((2 << 26) | (2 << 28) | (2U << 30));
```

- PB13 → MODER bits 27:26
- PB14 → MODER bits 29:28
- PB15 → MODER bits 31:30
- `2` = `10` = **Alternate Function mode**

⚠️ `3U << 30` এ `U` কেন?

- `3 << 30` = signed int shift, bit31 sign bit হয়ে UB (undefined behavior)
- `3U << 30` = unsigned shift, safe। Compiler warning এড়াতে।

```
GPIOB_AFRH  &= ~((0xF << 20) | (0xF << 24) | (0xFU << 28));
GPIOB_AFRH  |=  ((5 << 20) | (5 << 24) | (5U << 28));
```

- **AFRH** = AF register for pins 8-15
- PB13 → AFRH[23:20] (nibble 5)
- PB14 → AFRH[27:24] (nibble 6)
- PB15 → AFRH[31:28] (nibble 7)
- **`5` = AF5 = SPI2** (Datasheet AF table)

MISO pull-down (SPI-specific critical)

```
GPIOB_PUPDR &= ~(3 << 28);  GPIOB_PUPDR |= (2 << 28);          /* MISO pull-down */
```

- **PB14 (MISO) pull-DOWN**। কেন pull-up না?
- MISO = slave এর output (master এর input)
- Slave powered off/disconnected হলে MISO floating → random data
- **Pull-down দিয়ে idle LOW** force করা হয়

- এতে master garbage read করে "কিছু নেই" বুঝতে পারে

Contrast: UART এ RX pull-UP ছিল (idle HIGH)। SPI MISO pull-DOWN (idle LOW) — কারণ SPI idle state LOW।

PC13 button pull-up (আগের মতো)

```
GPIOC_PUPDR &= ~(3 << 26);  GPIOC_PUPDR |= (1 << 26);
```

SPI2_CR1 — সবচেয়ে important configuration

```
SPI2_CR1 = (1 << 2) | (5 << 3) | (1 << 9) | (1 << 8) | (1 << 6);   /* master */
```

একটা line এ ৫টা bit set — এক এক করে দেখি:

Bit	Field	Value	Meaning
2	**MSTR**	1	**Master mode**
5:3	**BR** (Baud Rate)	101 = 5	fPCLK/64 = 16MHz/64 = **250 kHz**
9	**SSM**	1	Software Slave Management
8	**SSI**	1	Internal NSS = HIGH (master active)
6	**SPE**	1	**SPI Enable**

বিস্তারিত:

`(1 << 2)` → MSTR=1 (Master)

- Master মানে **SCK এই board generate করবে**
- Slave code এ এই bit **0** থাকবে

`(5 << 3)` → BR=101 (fPCLK/64 = 250kHz)

- Baud Rate divider table:
  - 000 = /2, 001 = /4, 010 = /8, 011 = /16
  - 100 = /32, **101 = /64**, 110 = /128, 111 = /256
- 16MHz / 64 = **250kHz** — reliable speed for lab

`(1 << 9) | (1 << 8)` → SSM=1, SSI=1

- **SSM (Software Slave Management)** = NSS pin ignore, internal NSS ব্যবহার
- **SSI (Internal Slave Select)** = 1 → internal NSS HIGH (master as "selected")
- **কেন?** — If NSS pin floating LOW, master mode automatically disable হয়ে যায়। SSM+SSI দিয়ে pin এর ঝামেলা এড়ানো।

- `SSM=1, SSI=0` হলে **master mode disable হয়ে যাবে!** — exam এ tricky question

`(1 << 6)` → **SPE=1 (SPI Enable)**

Hardware on। এটা না দিলে SPI dead।

⚠️ কোন bit গুলো set হয়নি (default = 0)?

Bit	Field	Default	Meaning
0	CPHA	0	Clock phase = 0
1	CPOL	0	Clock polarity = 0 (**SPI mode 0**)
7	LSBFIRST	0	MSB first
11	DFF	0	8-bit data frame

SPI Mode 0 (CPOL=0, CPHA=0) = idle LOW, rising edge এ sample — সবচেয়ে common mode।

Slave এও একই mode (CPOL=0, CPHA=0) থাকতে হবে — না হলে bit misalign হবে।

5️⃣ Main Loop — Master Logic

State variables

```
int prev = 0, prevB = 0;
```

- `prev` = master button এর আগের state
- `prevB` = slave button এর আগের state (polling এ পেয়েছি)
- **Rising edge detect** এর জন্য

Part A — MY button → tell slave

```
int now = pressed();
if (now && !prev) {
    delay_ms(20);
    if (pressed()) {
        spi_xfer('S');                    /* tell slave: blink */
        delay_ms(SLAVE_BLINK_WAIT);       /* let it finish */
        if (spi_xfer(0x00) == 'D')        /* 'D' comes only if it really blinked
*/
            little_blink();               /* my ack */
        delay_ms(10);                     /* gap before next byte */
    }
```

```
}
prev = now;
```

Line by line:

1. **`now = pressed()`** → button এখন চাপা?
2. **`now && !prev`** → rising edge (debounced)
3. **`delay_ms(20)`** → debounce
4. **`pressed()`** → এখনো চাপা confirm
5. **`spi_xfer('S')`** → slave কে `'S'` পাঠাও — "আমি চাপ দিয়েছি, তুমি blink করো"
   SPI এ full-duplex, তাই **ইসকা** return value (slave এর reply) **এখনই ignore হয়** — cause slave এখনো `'S'` process করেনি
6. **`delay_ms(2500)`** → 2.5 sec অপেক্ষা। এই সময়ে slave blink(5) করছে (~2000ms)
7. **`spi_xfer(0x00)`** → dummy `0x00` পাঠাও, slave এর reply পড়ো:
   - Slave এখন reply হিসেবে `'D'` পাঠাবে (কারণ সে blink শেষ করে 'D' load করেছে)
   - `'D'` = "আমি সত্যিই blink করেছি"
   - Reply না পেলে (slave absent) → garbage
8. **`if (... == 'D')`** → **`little_blink()`** → confirmation
9. **`delay_ms(10)`** → পরের transfer এর আগে gap (slave এর TX register refill হওয়ার সময়)

🔑 **Critical design point:**

- `'S'` পাঠানোর সময় slave যে reply দেয়, সেটা ignore
- তারপর slave blink শেষ করে reply হিসেবে `'D'` load করে
- Master আবার dummy পাঠিয়ে `'D'` collect করে
- **Two-step handshake** — "ack-by-next-byte"

Part B — Poll slave's button

```
unsigned char r = spi_xfer(0x00);
int nowB = (r == 'B');
if (nowB && !prevB) {
    blink(5);
    spi_xfer('K');
    delay_ms(SLAVE_LITTLE_WAIT);
}
prevB = nowB;
```

1. **`spi_xfer(0x00)`** → dummy পাঠাও, slave এর reply পড়ো
   Slave reply = তার button state (`'B'` বা `0x00`)
2. **`nowB = (r == 'B')`** → slave pressed?
3. **`nowB && !prevB`** → slave button এর rising edge
4. **`blink(5)`** → আমার LED 5 বার (reaction)
5. **`spi_xfer('K')`** → slave কে `'K'` পাঠাও — "তোমার press আমি দেখেছি"
   Slave পরের transfer এ `'K'` receive করে `little_blink()` করবে
6. **`delay_ms(600)`** → slave little-blink (480ms) শেষ হওয়ার অপেক্ষা

7. **`prevB = nowB`** → state save

Loop delay

```
delay_ms(10);
```

Polling rate। প্রতি 10ms এ slave কে জিজ্ঞেস "তোমার button চাপা?"

🔄 Master Full Workflow Example

Example 1: Master button pressed

```
T=0ms     [MASTER] pressed()=1, rising edge
          spi_xfer('S') → slave RXNE, cmd='S'

T=1ms     [SLAVE] cmd=='S' → blink(5) শুরু, reply='D' load

T=1..2001ms  [SLAVE] LED blink ×5
             [MASTER] delay_ms(2500) — অপেক্ষা

T=2502ms  [MASTER] spi_xfer(0x00) → slave reply 'D' পেল
          little_blink() → 480ms

T=3000ms  পরের poll ready
```

Example 2: Slave button pressed (Slave locally presses)

```
T=0ms     [SLAVE] user button চাপে (local, bus event না)

T=10ms    [MASTER] spi_xfer(0x00) → slave reply 'B'
          nowB=1, prevB=0 → rising edge
          blink(5) → 2000ms
          spi_xfer('K') → slave cmd='K'

T=11ms    [SLAVE] cmd=='K' → little_blink() → 480ms

T=600ms   [MASTER] delay(600) শেষ, loop চালু
```

Example 3: Slave absent

```
[MASTER] spi_xfer(0x00) → MISO pull-down থাকায় 0x00 আসে
         r=0x00 != 'B' → ignore
         Loop চলতে থাকে, hang নেই ✅
```

🟢 PART 2: SLAVE Code — Line by Line

⚠️ Master vs Slave — main differences

Feature	Master	Slave
SPI2_CR1 MSTR bit	1 (master)	**0 (slave)**
Baud rate	Set (BR bits)	Ignore (master দেয়)
SSM/SSI	Both 1	**SSI=0** (selected when NSS low, কিন্তু software)
SCK	Generate	**Receive**
Init	Simple	**`spi_reset()` দিয়ে clean**
Preload	নেই	**TX register preload দরকার**

1️⃣ Extra register — `RCC_APB1RSTR`

```
#define RCC_APB1RSTR (*(volatile unsigned int *)0x40023820)
```

- **Reset register** — SPI2 কে hardware-level reset করতে
- bit14 set → SPI2 reset। bit14 clear → release
- Master এর এই register লাগে না, শুধু slave এর reset recovery এর জন্য

2️⃣ `spi_reset()` — Critical for slave

```
static void spi_reset(void) {
    RCC_APB1RSTR |=  (1 << 14);   // assert reset
    RCC_APB1RSTR &= ~(1 << 14);   // release reset
    SPI2_CR1 = (1 << 9) | (1 << 6);       /* slave, software NSS selected, enable
*/
}
```

Line by line:

1. **`RCC_APB1RSTR |= (1<<14)`** → SPI2 **reset assert** — সব internal state, shift register, buffer clear
2. **`RCC_APB1RSTR &= ~(1<<14)`** → Reset **release** — SPI2 clean অবস্থায় ফিরে আসে (CR1=0)

3. `SPI2_CR1 = (1<<9) | (1<<6)` → নতুন config:
    - **bit9 = SSM=1** → Software slave management
    - **bit6 = SPE=1** → SPI Enable
    - **MSTR bit (2) = 0** → **Slave mode** ← master এর থেকে main difference
    - **SSI bit (8) = 0** → Internal NSS LOW = "selected by master"
    - BR bits ignore (slave এ clock master দেয়)

কেন reset দরকার? — Slave এ bit desynchronization হতে পারে:

- Master যদি byte পাঠায় আর slave reply না দেয় → shift register এ mismatch
- Unknown byte এলে, বা slave reply late হলে → reset দিয়ে fresh start

---

3️⃣ Initialization

Clock enable, GPIO, PA5, PB13/14/15, PC13 — সব master এর মতোই।

MISO pull-down পার্থক্য

```
GPIOB_PUPDR &= ~((3 << 26) | (3U << 30));                    /* SCK + MOSI pull-
down: */
GPIOB_PUPDR |=  ((2 << 26) | (2U << 30));                    /* no fake clocks
when master is off */
```

- **PB13 (SCK) pull-down** ← master এর code এ ছিল না
- **PB15 (MOSI) pull-down** ← master এর code এ ছিল না
- **কেন?**
    - Master না থাকলে SCK/MOSI floating থাকলে slave random clock pulse পেতে পারে → fake transfer
    - Pull-down দিয়ে idle LOW force করা হয়
    - **"no fake clocks when master is off"** — comment এ ঠিকই বলা আছে

Slave CR1 init (spi_reset এর মধ্যে)

```
SPI2_CR1 = (1 << 9) | (1 << 6);       /* slave, software NSS selected, enable */
```

`bit9 SSM=1`, `bit6 SPE=1` — বাকি সব 0 → slave mode, mode 0

Preload TX register

```
while (!(SPI2_SR & (1 << 1)));                /* TXE */
SPI2_DR = 0x00;                               /* preload: not pressed */
```

🔑 **Critical for slave:** TX buffer এ **শুরু থেকেই কিছু থাকতে হবে**।

- Master প্রথম transfer এ dummy পাঠালে slave এর shift register এ already একটা byte থাকা দরকার

- **না থাকলে** → slave first byte হিসেবে garbage/previous value পাঠাবে
- এখানে `0x00` দিয়ে preload — "আমার button চাপা না"

4️⃣ Slave Main Loop — Event Driven

```
while (1) {
    while (!(SPI2_SR & (1 << 0)));                /* RXNE: transfer came */
```

- **`RXNE bit0` set হলে** একটা transfer complete হয়েছে
- **Master SCK generate করলেই** hardware এ byte shift হয়ে RXNE set হয়
- Slave **কখনো নিজে transfer শুরু করে না** — master এর অপেক্ষায়

```
    unsigned int  sr  = SPI2_SR;
    unsigned char cmd = (unsigned char)SPI2_DR;
```

- **`sr = SR`** → full status register snapshot। **DR পড়লে RXNE clear হয়**, কিন্তু SR এ অন্যান্য bits (TXE) এখনো check করা যাবে
- **`cmd = DR`** → master এর পাঠানো byte

```
    if (!(sr & (1 << 1))) spi_reset();            /* TXE=0: I'm one byte late */
```

🔑 এই line exam এর main conceptual question:

- **TXE bit1** = "TX buffer খালি"
- `TXE=0` মানে **আমার reply এখনো TX buffer এ** — master আগেই পরের byte পাঠিয়েছে, slave এক byte পিছিয়ে
- Bit desync! Shift register এ master/slave এর buffer misaligned
- **সমাধান**: `spi_reset()` → সব clear, fresh start

```
    unsigned char reply = pressed() ? 'B' : 0x00;  /* my button state */
```

- Default reply = আমার button state
- Master এর poll এ এটা পড়বে

```
    if      (cmd == 'S') { blink(5); reply = 'D'; }    /* master pressed → blink + ack */
    else if (cmd == 'K') little_blink();               /* my press acked → little blink */
    else if (cmd != 0x00) { delay_ms(1); spi_reset(); } /* unknown byte: bit sync lost */
```

কোন case এ কী:

1. **`cmd=='S'`** → master এর press → **আমি blink(5)** (2000ms) → **reply='D'** (ack)
2. **`cmd=='K'`** → master আমার press ack করেছে → **little_blink()** (480ms) → reply default (button state)

3. **`cmd != 0x00`** (e.g. `'X'`, `'D'`, garbage) → **unknown, bit sync lost** → `delay_ms(1)` দিয়ে settle, তারপর `spi_reset()`। **Master এর `'D'` wait করা reply হিসেবে আসা উচিত না** — যদি আসে, mane কিছু ভুল

```
while (!(SPI2_SR & (1 << 1)));                /* TXE */
SPI2_DR = reply;
```

- পরের transfer এর জন্য **reply preload**
- Master পরের dummy পাঠালে এই byte টা MISO দিয়ে যাবে
- **Preload না করলে** slave empty buffer পাঠাবে → garbage

🔄 Slave Workflow Example

Example: Master 'S' পাঠাল

```
T=0ms      [MASTER] spi_xfer('S')
           ↓ MOSI: 'S', SCK pulses

T=0.04ms   [SLAVE] 8 SCK পরে RXNE=1
           sr check: TXE=1 (preloaded 0x00 খালি হয়েছে, এখন empty)
           cmd='S'
           reply = pressed() ? 'B' : 0x00  // e.g. 0x00
           cmd=='S' → blink(5) 시작, reply='D'
           while(TXE) → SPI2_DR = 'D'  (preload)

T=0.05ms   [MASTER] RXNE=1, spi_xfer returns 0x00 (slave এর preloaded value)
           (এই 0x00 ignore হয়)

T=2001ms   [SLAVE] blink শেষ

T=2502ms   [MASTER] delay শেষ, spi_xfer(0x00)
           ↓ MOSI: 0x00, SCK pulses

T=2502.04ms [SLAVE] RXNE=1, cmd=0x00
              reply = 'B' or 0x00 (button state)
              cmd==0x00, unknown নয়, reset trigger না
              preload reply

T=2502.05ms [MASTER] RXNE=1, reads 'D' (আগের transfer এ preload ছিল)
              'D' == 'D' → little_blink()  ✓
```

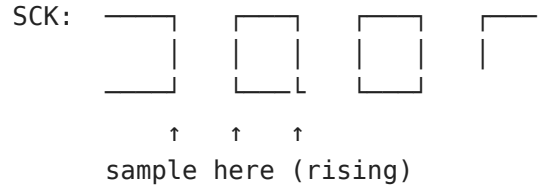
🔑 **Observation:** SPI এ সবসময় **one transfer delay**। Master যা পাঠায়, তার reply **পরের transfer এ** আসে। Master এটাই exploit করে — `'S'` পাঠিয়ে তারপর dummy পাঠিয়ে `'D'` collect করে।

🔍 Important Concepts — Exam Focus

Q1: SPI Mode 0 কী? কেন slave এও একই mode?

Mode 0 = CPOL=0, CPHA=0:

- **CPOL=0** → idle state LOW
- **CPHA=0** → **first (rising) edge এ sample**, second (falling) edge এ shift



Master আর slave দুইজনকেই একই mode এ থাকতে হবে — নইলে sample edge ভুল হয়ে bit misalign হবে।

Slave এর CR1 এ CPOL/CPHA default 0 → mode 0। Master এও same।

Q2: SSM আর SSI কেন?

- **SSM=1** → NSS pin বাদ, internal NSS
- **SSI** = internal NSS value
  - **Master: SSI=1** → internally "selected", master mode stays active
  - **Slave: SSI=0** → internally "not selected by default, master নির্বাচন করবে"
- **SSI=0 with SSM=1 in master mode** → master mode automatically disable হয়ে যায়! **Trap question** in exam।

Q3: `spi_xfer(0x00)` এ `0x00` পাঠানো কেন?

SPI এ data "receive" করতে হলে **master কে clock pulse generate করতে হয়**। Clock generate করার একমাত্র উপায় = কিছু পাঠানো। তাই `0x00` (dummy) পাঠানো হয় — slave এটা ignore করে, কিন্তু এর ফলস্বরূপ SCK pulse generate হয়ে slave এর reply master এর দিকে আসে।

Q4: `spi_reset()` slave এ কেন আর master এ নেই?

Master active controller — সে নিজেই timing জানে, sync হারায় না (কারণ SCK তার control এ)। **Slave passive** — master যেকোনো সময় byte পাঠাতে পারে। যদি slave এর reply ready না থাকে, shift register desync হয়। Reset দিয়ে fresh start দরকার।

Q5: TXE=0 হলে slave reset কেন?

`TXE=0` মানে slave এর TX buffer এখনো ভরা → **আগের reply পাঠানো হয়নি**।

- Master আগেই পরের byte পাঠিয়ে দিয়েছে
- Slave এক byte পিছিয়ে পড়েছে (shift register misaligned)
- আরেক byte এলে আরও misalign হবে
- **Reset** করলে shift register, TX buffer সব clear → fresh start

Q6: Master কেন `SLAVE_BLINK_WAIT 2500ms` wait করে?

Slave `blink(5)` = 5 × (200 + 200) = 2000ms।

- এই 2000ms এ slave একটা loop এ ব্লকড, TX/MISO handle করছে না
- Master যদি এর মধ্যে dummy পাঠায়, slave response দিতে পারবে না
- 2500ms = 2000ms + margin → slave finished, ready

Q7: Master `'D'` পায় কীভাবে — slave তো শুধু `reply` load করে?

Two-transfer handshake:

1. Master: `spi_xfer('S')` → slave receives, `reply='D'` load করে buffer এ
2. Master: `delay_ms(2500)` → slave blink শেষ
3. Master: `spi_xfer(0x00)` → slave buffer থেকে `'D'` shift out → master পায়

One-byte delayed ack mechanism — exam এ এই concept emphasize করো।

Q8: Unknown byte এলে `delay_ms(1)` কেন?

`spi_reset()` করার আগে 1ms delay:

- Master এর SCK এখনো pulse দিচ্ছে কিনা দেখার জন্য
- Reset করার সময় SCK active থাকলে reset কার্যকর না হতে পারে
- 1ms এ SCK line settle হয় → নিরাপদ reset

📊 Master vs Slave Side-by-Side

Aspect	Master	Slave
MSTR bit (CR1 bit2)	1	**0**
BR bits (5:3)	101 (=/64)	ignore (any)
SSM (bit9)	1	1
SSI (bit8)	**1**	**0**

Aspect	Master	Slave
SPE (bit6)	1	1
SCK role	Generate	Receive
Init complexity	Simple	Reset + preload
TX preload	না	**হ্যাঁ (essential)**
Error recovery	none	`spi_reset()`
Pull-down	PA10 (না) / MISO?	**SCK+MOSI+MISO pull-down**
Main loop	Poll + button	Only respond to RXNE

🛠️ Full Workflow — Master + Slave Combined

```
┌──────────────────────────────────────────────────────────────┐
│  Master button pressed                                       │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  MASTER                          SLAVE                       │
│  ──────                          ─────                       │
│  Button → rising edge                                        │
│  spi_xfer('S') ──SCK+MOSI────►   RXNE, cmd='S'               │
│                                   blink(5) start (2000ms)    │
│                                   reply='D' preload          │
│  delay(2500) — wait               blink continues...         │
│                                   blink ends (2000ms)        │
│  spi_xfer(0x00) ──SCK+MOSI────►   RXNE, cmd=0x00             │
│                                   reply='B'/'0x00' preload   │
│  Read MISO ◄─────MISO──────────── 'D' (from previous preload) │
│  'D' == 'D' → little_blink() ✓                               │
│                                                              │
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
│  Slave button pressed                                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  MASTER (polling)                SLAVE                       │
│  ────────────────                ─────                       │
│                                   User button pressed        │
│  spi_xfer(0x00) ──SCK+MOSI────►   RXNE, cmd=0x00             │
│                                   pressed()==1 → reply='B'   │
│  Read MISO ◄─────MISO──────────── 'B' (from previous preload) │
│  r=='B' → rising edge                                        │
```

```
│  blink(5) → 2000ms                                              │
│  spi_xfer('K') ──SCK+MOSI────►    RXNE, cmd='K'                 │
│                                    little_blink() → 480ms       │
│  delay(600) — wait                                              │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

📝 Quick Cheat Sheet — Exam এর আগে

```
SPI MASTER INIT:
1. RCC: GPIOA+B+C, SPI2 (APB1ENR bit14)
2. PA5 output, PB13/14/15 = AF5
3. PB14 (MISO) pull-down
4. PC13 pull-up
5. CR1 = MSTR(2) | BR=5(5<<3) | SSM(9) | SSI(8) | SPE(6)

SPI SLAVE INIT:
1. RCC: same + APB1RSTR
2. GPIO same, PB13(SCK)/PB15(MOSI)/PB14(MISO) pull-down
3. PC13 pull-up
4. spi_reset(): RSTR toggle → CR1 = SSM(9) | SPE(6)
5. Preload TX: TXE wait, DR=0x00

XFER:
  while(!TXE); DR=b;
  while(!RXNE); return DR;

MASTER MAIN:
- button rising edge → spi_xfer('S') → delay(2500) →
  if (spi_xfer(0x00)=='D') little_blink()
- poll: r=spi_xfer(0x00); if(r=='B' && !prevB):
  blink(5); spi_xfer('K'); delay(600)
- delay(10)

SLAVE MAIN:
- while(!RXNE)
- sr=SR; cmd=DR;
- if(!(sr&TXE)) spi_reset();   // one byte late
- reply = pressed() ? 'B' : 0
- 'S' → blink(5), reply='D'
- 'K' → little_blink()
- else if (cmd!=0x00) → delay(1); spi_reset();
- preload reply

CR1 keys:
  bit2  MSTR (1=master, 0=slave)
  bit3-5 BR (baud rate divider)
  bit6  SPE (enable)
  bit8  SSI (internal NSS — 1 master, 0 slave)
  bit9  SSM (software slave mgmt)
```

```
SR keys:
  bit0 RXNE (rx not empty)
  bit1 TXE  (tx empty)
  bit7 BSY  (busy)

BR lookup (bit5:3):
  000=/2, 001=/4, 010=/8, 011=/16,
  100=/32, 101=/64, 110=/128, 111=/256

Baud = fPCLK / divider = 16MHz / 64 = 250kHz
```

🎓 Exam এ যে প্রশ্ন গুলো আসতে পারে

1. **"SPI আর I2C এর প্রধান পার্থক্য কী?"** → SPI 3-wire (SCK/MOSI/MISO), full-duplex, no address, master generates clock. I2C 2-wire, half-duplex, address-based।
2. **"SPI এ MISO pull-down কেন, আর UART এ RX pull-up কেন?"** → SPI idle state LOW → pull-down। UART idle state HIGH → pull-up।
3. **"`spi_xfer(0x00)` এ 0x00 না দিলে কী হবে?"** → SCK generate হবে না → slave reply আসবে না। কারণ SPI এ receiving = sending dummy.
4. **"Master এ SSI=0 হলে কী হবে?"** → NSS internally LOW = "not selected" → master mode auto-disable → SPI কাম করবে না।
5. **"Slave এ TXE=0 হলে কেন reset?"** → এক byte late হয়ে গেছে → shift register desync। Reset দিয়ে recover.
6. **"Master কেন 2500ms wait করে 'S' এর পর?"** → Slave 5× blink (2000ms) এ busy → এর মধ্যে transfer হলে slave response দিতে পারবে না।
7. **"Two boards এ একই code চালানো যাবে SPI তে?"** → না। MSTR bit আলাদা (1 vs 0), SSI আলাদা (1 vs 0), slave এ preload+reset লাগে।
8. **"SPI full-duplex কেন?"** → MOSI আর MISO আলাদা wire। SCK pulse এ একইসাথে দুই দিকে bit shift হয়। (এখানে code polling-based, কিন্তু hardware capable)

শুভকামনা lab exam এর জন্য! 🎓

মূল কথা মনে রাখো:

- **SPI = shift register + clock + full-duplex**
- **Sending = Receiving** (একই transfer এ দুটোই)
- **Master drives clock, Slave follows**
- **`spi_xfer` = 1 byte out + 1 byte in (always)**
- **Slave এ preload + reset = critical**

কোনো specific aspect (BAUD calculation, CPOL/CPHA mode, অথবা interrupt-based SPI version) গভীরে বুঝতে চাইলে বলো — আরও detail করে দেবো।

akhon bolo konta kokhon use korte hobe, kon situation a ?

কোন Protocol কখন ব্যবহার করবে? — Real-World Decision Guide

Exam এ প্রায়ই আসে: **"এই situation এ কোন protocol use করবে?"** চলো একদম practical ভাবে দেখি।

🎯 এক নজরে Decision Table

Situation	Protocol	কেন
Sensor অনেকগুলো, তার কম	**I2C**	Address দিয়ে চেনা যায়
দ্রুত data (SD card, LCD)	**SPI**	MHz range
PC ↔ MCU	**UART**	Simple, PC এ COM port
দূরের device (কয়েক মিটার)	**UART** (RS-485 হলে ভালো)	Long distance
Same board এ দুটো MCU	**SPI বা UART**	Depends on speed
এক wire এ 2+ device	**I2C**	Address support
Two-way, clock-driven, fast	**SPI**	Full duplex + speed
WiFi/Bluetooth module	**UART**	Standard interface
EEPROM/FRAM	**I2C**	Small data, addressable
SD card, Flash memory	**SPI**	Bulk data, fast
Debug console	**UART**	printf style
Display (OLED, TFT)	**SPI** (TFT) / **I2C** (OLED)	Speed vs pins
RTC chip	**I2C**	Small register read/write
ADC chip	**SPI**	High speed sample
GPS module	**UART**	Standard NMEA output

🔵 I2C কখন ব্যবহার করবে?

✅ যখন এই condition গুলো match করে:

1. **অনেক device, কম pin** — শুধু 2 wire (SCL+SDA), 100+ device possible
2. **Slow speed ঠিক আছে** — 100kHz / 400kHz, sensor এর জন্য যথেষ্ট
3. **Address দিয়ে select করতে চাও** — প্রতিটা device এর unique address
4. **Bidirectional but sequential** — half-duplex, কিন্তু control + data mix
5. **Device power management** — device hot-plug এ সমস্যা কম
6. **Multi-master possible** — একাধিক master একই bus এ

🎯 Real Example situations:

Example 1: Temperature + Humidity + Pressure sensor

```
MCU ──┬── BME280 (0x76)
      ├── SHT31  (0x44)
      └── BMP280 (0x77)
```

শুধু 2 wire, 3 sensor, সব address আলাদা → **I2C perfect**

Example 2: EEPROM read/write

```
MCU ── AT24C256 EEPROM (0x50)
```

Small data, register-based → **I2C**

Example 3: OLED display (SSD1306)

```
MCU ── SSD1306 OLED (0x3C)
```

128×64 ছোট display, refresh rate কম → **I2C চলে** (SPI হলে দ্রুততর)

❌ I2C ব্যবহার করবে না যখন:

- **High-speed data** দরকার (video, SD card)
- **Long distance** (I2C মাত্র 1m এর মধ্যে ভালো)
- **Noise-heavy environment** (I2C তুলনামূলক sensitive)
- **Single device only** — তাহলে SPI বা UART ভালো (I2C complexity বাড়তি)

🟢 SPI কখন ব্যবহার করবে?

✅ যখন এই condition:

1. **Speed critical** — 1MHz থেকে 100MHz পর্যন্ত possible
2. **Large data transfer** — display, SD card, Flash memory

3. **Full-duplex দরকার** — একইসাথে send + receive
4. **Point-to-point বা few slaves** — প্রতিটা slave এর আলাদা NSS pin (pin cost!)
5. **Real-time streaming** — ADC, DAC, sensor stream
6. **Noise immunity** চাও — push-pull output, differential হলে আরও ভালো

🎯 Real Example situations:

Example 1: TFT LCD display (ILI9341)

```
MCU ──SPI── ILI9341 (320×240 color)
```

Pixel data bulk transfer → **SPI must** (I2C হলে 100× slow)

Example 2: SD card

```
MCU ──SPI── MicroSD card
```

File read/write MB-range → **SPI only**

Example 3: High-speed ADC (MCP3008)

```
MCU ──SPI── MCP3008 (8-channel 10-bit ADC)
```

Sample rate 200ksps → **SPI**

Example 4: External Flash (W25Q128)

```
MCU ──SPI── W25Q128 (16MB Flash)
```

Firmware storage, data logging → **SPI**

Example 5: Wireless module (nRF24L01)

```
MCU ──SPI── nRF24L01 (2.4GHz transceiver)
```

Packet stream, low latency → **SPI**

❌ SPI ব্যবহার করবে না যখন:

- **অনেক slave** (প্রতিটা NSS = 1 pin; 10 slave = 10 pin!)
- **Long distance** (SPI মাত্র few cm, board-to-board)
- **Hot-plug** দরকার (SPI hot-plug support নেই)
- **শুধু 2 device, distance moderate** — UART simpler

🟡 UART কখন ব্যবহার করবে?

✅ যখন এই condition:

1. **Point-to-point communication** — শুধু 2 device
2. **PC ↔ MCU** — USB-serial দিয়ে debug/console
3. **Standard module interface** — GPS, WiFi, Bluetooth, GSM
4. **Long distance** (RS-232/RS-485) — কয়েক মিটার
5. **Simple + low pin** — 2 wire (TX+RX)
6. **Asynchronous acceptable** — clock share করা লাগবে না
7. **Bidirectional control** — AT commands, terminal

🎯 Real Example situations:

Example 1: Debug console / printf

```
MCU ──UART── USB-Serial ── PC (PuTTY)
```

printf দিয়ে debug → **UART must**

Example 2: ESP8266 WiFi module

```
MCU ──UART── ESP8266
```

AT commands, standard interface → **UART**

Example 3: HC-05 Bluetooth

```
MCU ──UART── HC-05
```

Serial data transparent → **UART**

Example 4: GPS module (NEO-6M)

```
MCU ──UART── NEO-6M
```

NMEA sentence stream → **UART**

Example 5: PC ↔ MCU (USB Serial)

```
PC ──USB── CH340 ──UART── STM32
```

Standard COM port → **UART**

Example 6: Two MCU কয়েক মিটার দূরে

```
MCU1 ──RS-485── MCU2 (10m apart)
```

Long distance → **UART over RS-485**

❌ UART ব্যবহার করবে না যখন:

- **একাধিক device** (UART এ address নেই)
- **High speed** (সাধারণত 115200 max, high speed হলে unreliable)
- **Bus topology** দরকার (UART point-to-point only)
- **Synchronized clock** দরকার (UART asynchronous, baud mismatch সমস্যা)

🎬 Real Project — Decision Examples

Project 1: Weather Station

```
Requirements:
- BME280 (temp/humidity/pressure)
- OLED display (128×64)
- PC log via USB

Decision:
- BME280  → I2C (address 0x76, slow data)
- OLED    → I2C (0x3C, small display)
- PC log  → UART (USB-serial)

কেন? সব low-speed, pin কম, address-based
```

Project 2: Data Logger

```
Requirements:
- SD card (write data every 1s)
- RTC (timestamp)
- High-speed ADC (audio, 44.1kHz)
- Display (TFT)

Decision:
- SD card → SPI (MB speed)
- RTC     → I2C (small register)
- ADC     → SPI (audio rate)
- TFT     → SPI (pixel data)
- Debug   → UART

কেন? Speed-critical গুলো SPI, simple register গুলো I2C
```

Project 3: Home Automation

```
Requirements:
- 5 temperature sensors (different rooms, same bus)
- Central MCU
- WiFi to cloud

Decision:
```

```
- Sensors  → I2C (multi-drop, address each)
- WiFi     → UART (ESP8266, AT commands)

কেন? Sensors অনেক, address দরকার → I2C। WiFi module standard UART
```

Project 4: Two MCU Short Distance

```
Requirements:
- 2 STM32 boards
- Both need to talk
- Sync speed

Decision:
- Option A: SPI (fast, but master/slave code আলাদা)
- Option B: UART (simple, symmetric code)

যদি speed critical → SPI
যদি simplicity wa̍ি → UART (এই lab এর UART two-way switch perfect example!)
```

🔍 Advanced Decision Criteria

1️⃣ Pin Count Matters?

```
Device count × pin per device:
- UART: 2 pin per peer (TX+RX) — always
- I2C:  2 pin total (SCL+SDA) — address-based!
- SPI:  3+ pin per slave (SCK+MOSI+MISO+NSS)

Example: 8 sensors সংযোগ:
- UART: 8 × 2 = 16 pin (impossible!)
- I2C:  2 pin (best!)
- SPI:  3 + 8×NSS = 11 pin (wasteful)
```

→ অনেক sensor = I2C

2️⃣ Distance Matters?

```
Range:
- SPI:  board-to-board (10cm)
- I2C:  1m (low speed), 10m (with buffer IC)
- UART: 15m (RS-232), 1200m (RS-485)

Example: 10m দূরে sensor:
- SPI: ❌ কাজ করবে না
- I2C: ⚠️ borderline
- UART: ✅ RS-485 হলে perfect
```

3️⃣ Data Rate Matters?

```
Speed:
- UART:  9600 bps – 1 Mbps
- I2C:   100 kbps – 3.4 Mbps
- SPI:   1 Mbps – 100 Mbps

Example: 1920×1080 image (6MB):
- UART: 6MB / 1Mbps = 48 sec ❌
- I2C:  6MB / 3.4Mbps = 14 sec ❌
- SPI:  6MB / 50Mbps = 1 sec ✅
```

→ Bulk data = SPI

4️⃣ Noise Immunity Matters?

```
Immunity:
- UART:  RS-485 differential হলে best
- I2C:   moderate (external pull-up)
- SPI:   push-pull (strong), কিন্তু many wires

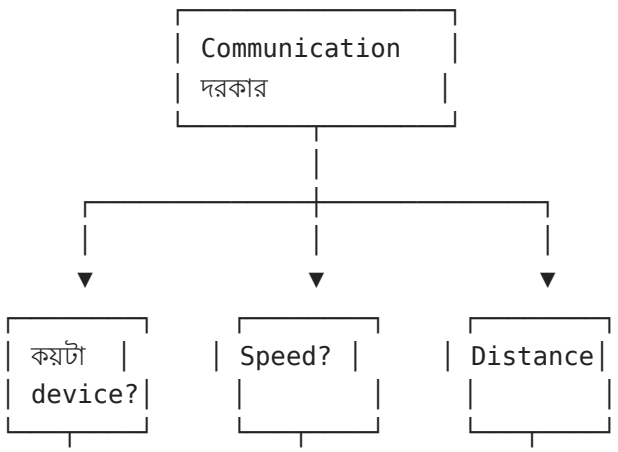
Industrial (motor noise):
- Sensors via RS-485 (UART)
- Display via SPI (fast, on-board)
```

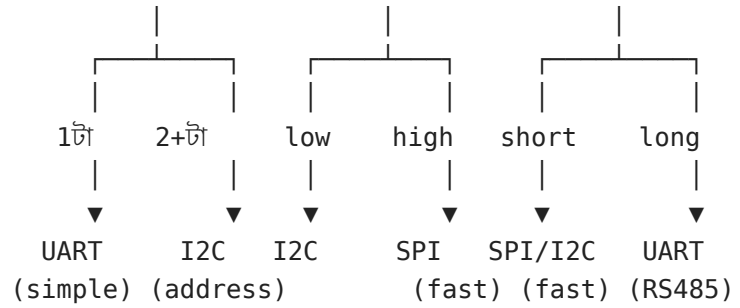
5️⃣ Clock Sync Matters?

```
Sync requirement:
- UART: asynchronous, baud mismatch এ ভুল data
- I2C:  synchronous, clock embedded
- SPI:  synchronous, clock embedded, fastest

Example: microsecond timing requirement → SPI/I2C
Example: relaxed timing → UART OK
```

📊 Decision Flowchart (Exam এ লিখো)





🎓 Lab Exam এ এই Table মনে রাখো

Scenario	Answer	Reason
"5 sensor থেকে data পড়তে হবে"	**I2C**	address-based, কম pin
"SD card এ file write"	**SPI**	fast bulk data
"PC এ data পাঠাতে হবে"	**UART**	standard COM port
"2 board, দ্রুত sync"	**SPI**	clock-driven
"2 board, simple"	**UART**	symmetric code
"EEPROM থেকে data"	**I2C**	address-based register
"TFT display drive"	**SPI**	pixel speed
"GPS module"	**UART**	standard interface
"1 meter দূরে sensor"	**I2C**	acceptable range
"10 meter দূরে device"	**UART (RS-485)**	long distance
"শুধু 2 device, few cm"	**SPI/UART**	simple point-to-point
"শুধু 2 device, 10cm, synchronized data"	**SPI**	clock sync
"1 sensor, 2 pin বাকি"	**UART**	simplest
"5 sensor, 2 pin বাকি"	**I2C**	multi-device
"5 sensor, 10 pin বাকি"	**SPI**	fast+multi
"Battery powered, long idle"	**UART/I2C**	SPI pulls current

💡 Exam এ Tip — কীভাবে উত্তর দেবে

Question: "তোমার project এ 3টা temperature sensor আর একটা OLED display আছে। কোন protocol ব্যবহার করবে?"

Good Answer (exam এর জন্য):

> "আমি **I2C** ব্যবহার করব। কারণ:
>
> 1. **Multi-device**: 3টা sensor + 1টা display = 4টা device, প্রতিটার unique address আছে। I2C তে 2 wire এ সব connect করা যায়। SPI হলে প্রতি device এ NSS pin আলাদা লাগত (4 extra pin)।
> 2. **Speed যথেষ্ট**: Temperature sensor এর update rate কম (1Hz), OLED 128×64 কে 400kHz এ handle করা যায়। High speed দরকার নেই।
> 3. **Pin efficiency**: শুধু SCL+SDA+2 pull-up = 2 pin। SPI হলে 3 + 4 NSS = 7 pin।
> 4. **Simplification**: সব device একই bus এ, hot-swap possible।
>
> SPI ব্যবহার করতাম যদি display বড় হত (TFT) বা high-speed ADC থাকত। UART ব্যবহার করতাম যদি একটাই sensor হত।"

🎯 Final Cheat — "কোনটা কখন?" memory trick

```
┌──────────────────────────────────────────┐
│  MANY DEVICES          → I2C    (address) │
│  FAST DATA             → SPI    (clock)   │
│  TO/FROM PC            → UART   (COM)     │
│  LONG DISTANCE         → UART   (RS-485)  │
│  SIMPLE 2-WAY          → UART   (symmetry)│
│  HIGH-SPEED CONTROL    → SPI    (fast)    │
│  SMALL REGISTERS       → I2C    (small)   │
│  BULK STORAGE          → SPI    (bulk)    │
│  STANDARD MODULE       → UART   (universal)│
│  EXTRA 2 PINS ONLY     → UART   (minimal) │
│  EXTRA 2 PINS, MANY    → I2C    (bus)     │
│  EXTRA 3 PINS, FEW     → SPI    (speed)   │
└──────────────────────────────────────────┘
```

Mantra:

- **I2C = "Many, Small"** (many devices, small data)
- **SPI = "Fast, Big"** (fast speed, big data)
- **UART = "Simple, Standard"** (simplest, standard modules)

শুভকামনা exam এ! 🎓 এই decision criteria গুলো যদি exam এ লিখতে পারো — scenario-based প্রশ্নে পুরো নম্বর পাবে। কোনো specific scenario নিয়ে practice করতে চাইলে বলো — আমি

mock question দেবো।

3 ta code er logic gulo to bujhtecina, work flow kivabe hosse , scinario er moto kore real life a device gulo kivabe button press, release er sathe led gulor behavior er sathe relation hosse details bolo , egulo details example diye ekta ekta kore venge venge bolo please , full form gulo bolo 3 ta code a use hoyese
serially code line example diye bolo : I2C, UART,SPI
data packet basic format soho bolo please , kalk exam

🎯 I2C, UART, SPI — Complete Deep Dive (Exam Ready)

চলো একদম শূন্য থেকে শুরু করি — **real life device behavior** দিয়ে, তারপর code, তারপর packet format। প্রতিটা protocol আলাদা করে, example দিয়ে, line-by-line।

📖 PART 0: Full Forms + Basic Concepts (প্রথমে এটা বুঝো)

Full Forms (exam এ লিখতে হবে)

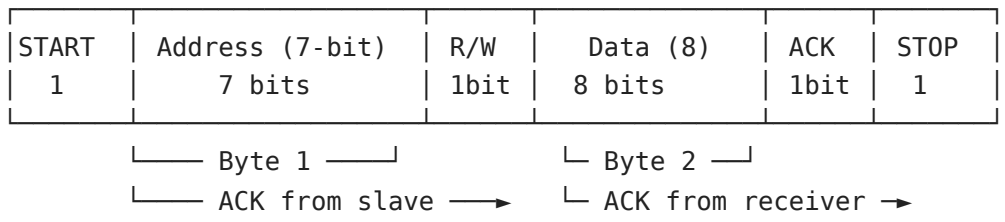
Abbreviation	Full Form	মানে
I2C	**Inter-Integrated Circuit**	দুই IC এর মধ্যে communication
UART	**Universal Asynchronous Receiver/Transmitter**	Clock ছাড়া serial communication
SPI	**Serial Peripheral Interface**	Peripheral device দের সাথে serial link
SCL	Serial **Clock** Line (I2C)	Clock wire
SDA	Serial **Data** Line (I2C)	Data wire
SCK	Serial **Clock** (SPI)	Clock wire
MOSI	**Master Out Slave In**	Master → Slave data
MISO	**Master In Slave Out**	Slave → Master data
NSS	**N**ot **S**lave **S**elect (also CS/SS)	Chip select

Abbreviation	Full Form	মানে
TX	**Transmit**	পাঠানোর pin
RX	**Receive**	Receive করার pin
ACK	**Acknowledge**	"পেয়েছি" signal
NACK/NAK	**Negative Acknowledge**	"পাইনি" signal
START	I2C start condition	SDA high→low during SCL high
STOP	I2C stop condition	SDA low→high during SCL high
DR	**Data Register**	Data রাখার register
SR	**Status Register**	Hardware status
CR1	**Control Register 1**	Configuration
BRR	**Baud Rate Register** (UART)	Speed control
CCR	**Clock Control Register** (I2C)	I2C speed
TRISE	**Rise Time** (I2C)	Signal rise time
OAR1	**Own Address Register 1** (I2C slave)	নিজের address
AF	**Alternate Function** (GPIO)	Pin এর special function
AFR	**Alternate Function Register**	AF select
ODR	**Output Data Register**	Pin output value
IDR	**Input Data Register**	Pin input value
MODER	**Mode Register**	Pin mode (in/out/AF)
PUPDR	**Pull-Up/Pull-Down Register**	Internal resistor
AHB1	**Advanced High-performance Bus 1**	Fast peripheral bus
APB1/APB2	**Advanced Peripheral Bus**	Slow peripheral bus
RCC	**Reset and Clock Control**	Clock/Reset generator

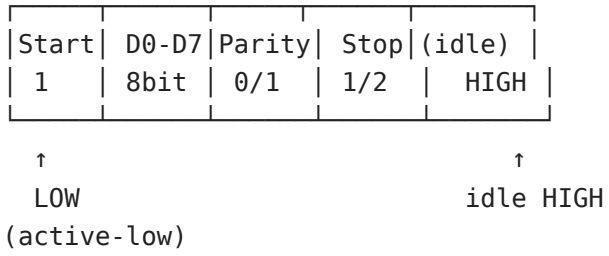
Abbreviation	Full Form	মানে
ENR	**Enable Register**	Clock on/off
RSTR	**Reset Register**	Peripheral reset
TXE	**Transmit Empty**	TX buffer empty flag
RXNE	**Receive Not Empty**	RX data ready flag
TC	**Transmission Complete**	TX fully done
SB	**Start Bit** (I2C)	START sent flag
ADDR	**Address matched** (I2C)	Address match flag
BTF	**Byte Transfer Finished** (I2C)	Byte done flag
TRA	**Transmitter/Receiver** (I2C)	Direction flag
BUSY	Bus busy	I2C busy flag
PE	**Peripheral Enable**	On/off switch
MSTR	**Master**	Master mode bit
SPE	**SPI Enable**	SPI on/off
SSM	**Software Slave Management**	Software NSS
SSI	**Software Slave Select Internal**	Internal NSS value
CPOL	**Clock Polarity**	Clock idle state
CPHA	**Clock Phase**	Sample edge

🔑 Basic Packet Format — তিনটা protocol এর তুলনা

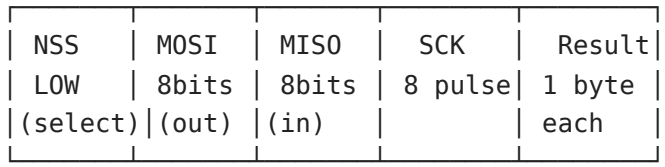
I2C Packet Format



UART Packet Format (8N1)



SPI Packet Format



🎬 PART 1: I2C — Real Life Story দিয়ে শুরু

📖 Real Life Setup

তুমি দুটো বন্ধু — রহিম (Master) আর করিম (Slave)। দুজন দুই ঘরে থাকো। একটা **একটাই কলিং বেল (SDA)** আর একটা **একটাই ঘড়ি (SCL)** আছে দুই ঘরের মাঝে।

নিয়ম (I2C Protocol):

- **ঘড়ি (SCL) সবসময় রহিম বাজায়** (master clock দেয়)
- **কলিং বেল (SDA) দিয়ে দুইজন কথা বলে** (data)
- **রহিম ডাকতে হবে** → বেল বাজিয়ে করিম এর নাম বলে
- **করিম নিজে কখনো ডাকতে পারবে না** — শুধু রহিমের ডাকের উত্তর দেয়
- **একসাথে দুইজন কথা বলতে পারবে না** (half-duplex)

🎭 Real Life Scenario 1: রহিম (Master board) এর button চাপা

🏠 Physical setup



🎬 ঘটনার ক্রম

T=0ms: রহিম তার button চাপে

```
রহিমের ঘর:                    করিমের ঘর:
🔘 Button চাপা                 🔘 Button চাপা না
PC13 = LOW (0V)                PC13 = HIGH (3.3V, pull-up)
💡 LED off                     💡 LED off
```

T=0ms: রহিমের code চেক করে `pressed() == 1` (true)

রহিম ভাবে: "আমার button চাপা হয়েছে! করিম কে বলি।"

T=0-20ms: রহিম debounce করে (20ms wait, আবার check)

```
যদি এখনো চাপা → real press
যদি ছেড়ে দিয়েছে → bounce (ignore)
```

T=20ms: রহিম করিম কে message পাঠায়

```
রহিম বেল (SDA) বাজায়:
START condition → "শোনো করিম"
Address 0x28 পাঠায় → "করিম, তুমি?"
R/W = 0 (write) → "আমি কিছু পাঠাচ্ছি"
Data = 'S' (0x53) → "আমি চাপ দিয়েছি"
STOP condition → "কথা শেষ"
```

T=21ms: করিম message পায়

```
করিমের code: "ADDR flag set" → "আমাকে ডাকা হয়েছে"
SR2 read → TRA=0 (write) → "রহিম আমাকে কিছু পাঠাচ্ছে"
cmd = 'S' পড়ল
cmd=='S' → blink(5) শুরু
```

T=21-2021ms: করিমের LED ৫বার blink

```
করিমের LED state:
T=21:    ON  → 200ms
T=221:   OFF → 200ms
T=421:   ON  → 200ms
T=621:   OFF → 200ms
T=821:   ON  → 200ms
T=1021:  OFF → 200ms
T=1221:  ON  → 200ms
T=1421:  OFF → 200ms
T=1621:  ON  → 200ms
T=1821:  OFF → 200ms
T=2021:  blink complete!
```

T=21ms: রহিম delay(2500ms) শুরু

```
রহিম ভাবে: "করিম কে সময় দিই blink শেষ করতে"
রহিমের LED off (কিছু করছে না)
```

T=2521ms: রহিম little_blink() করে (2×120ms)

```
রহিমের LED state:
T=2521: ON  → 120ms
T=2641: OFF → 120ms
T=2761: ON  → 120ms
T=2881: OFF → 120ms (শেষ)

রহিম ভাবে: "আমার কাজ শেষ, করিম কে বলেছি, নিজে ack দেখালাম"
```

🏁 Final Result

```
রহিম (MASTER):      করিম (SLAVE):
🔘 button: চাপা ছিল → ছেড়ে দিয়েছি
💡 LED: 2 বার ছোট blink  (ack)
                    🔘 button: চাপা না (নিজের)
                    💡 LED: 5 বার বড় blink (response)
```

কেন এই behavior? — "আমি চাপ দিয়েছি" signal পাঠানোর পর "তুমি পেয়েছ" confirmation পাওয়া। Rahimaর 2-blink মানে "করিম blink করেছে, আমি নিশ্চিত"।

🎭 Real Life Scenario 2: করিম (Slave) এর button চাপা

🎬 ঘটনার ক্রম

T=0ms: করিম তার button চাপে

```
করিমের ঘর:                    রহিমের ঘর:
🔘 Button চাপা                 🔘 Button চাপা না
PC13 = LOW                     PC13 = HIGH
💡 LED off                     💡 LED off

⚠️ করিম নিজে কিছু পাঠাতে পারবে না!
কারণ সে slave। রহিমের জিজ্ঞেস করার অপেক্ষায়।
```

T=0ms: রহিমের code polling করে

```
রহিম ভাবে: "করিমের button চেক করি"
i2c_read(0x28, &r) কল করে:

রহিম বেল (SDA) বাজায়:
START → "করিম, তোমার button কী অবস্থা?"
Address 0x28 + R/W=1 (read)
করিম reply পাঠায়
STOP
```

T=1ms: করিম address দেখে

```
করিমের code: "ADDR flag set"
SR2 read → TRA=1 (master reading)
```

```
TXE wait
I2C1_DR = pressed() ? 'B' : 0x00
            ↑
          button চাপা → 'B' (0x42) পাঠাল
```

করিম ভাবে: "রহিম আমার button চাইছে, আমি 'B' (চাপা) জানাই"

T=2ms: রহিম `r = 'B'` পায়

```
রহিমের code:
r = 'B'
nowB = (r == 'B') = 1
prevB = 0 (আগে চাপা না)
nowB && !prevB → rising edge! (new press)
→ blink(5) শুরু
→ i2c_write(0x28, 'K') → "করিম, দেখেছি"
```

T=2-2002ms: রহিমের LED ৫বার blink

```
রহিমের LED: ON/OFF ×5 → 2000ms
```

T=2ms: রহিম 'K' write করে, করিম পায়

```
করিমের code: ADDR, TRA=0 (write)
cmd = 'K'
cmd=='K' → little_blink() (2×120ms)
```

T=2-482ms: করিমের LED ২বার ছোট blink

```
করিমের LED: ON/OFF ×2 → 480ms
```

🏁 Final Result

```
রহিম (MASTER):          করিম (SLAVE):
🔘 button: চাপা না       🔘 button: চাপা ছিল
💡 LED: 5 বার বড় blink   💡 LED: 2 বার ছোট blink (ack)
(কারণ সে poll করে জেনেছে)  (কারণ she press করেছে)
```

মজার ব্যাপার: করিম তার button চাপল, কিন্তু **message পাঠাতে পারল না**। রহিম polling করে "ধরা পড়ল"। এটাই I2C এর slave limitation।

💻 Code — Line by Line with Real Explanation

Master code — `i2c_write(0x28, 'S')`

```
static int i2c_write(unsigned char addr, unsigned char data) {
```

Full form: `addr` = target address (0x28), `data` = byte to send ('S')

```
    unsigned int g;
    g = GUARD; while ((I2C1_SR2 & (1 << 1)) && --g); if (!g) return 0;
```

- `SR2 bit1 = BUSY` → বাস busy কিনা চেক
- Real life: "বেল বাজাচ্ছে কেউ? তাহলে wait করি"
- GUARD: 200000 বার চেক করে, শেষ হলে return 0 (no slave)

```
    I2C1_CR1 |= (1 << 8);
```

- `CR1 bit8 = START` → START condition
- Real life: **রহিম বেল বাজায়** → "শোনো সবাই!"

START condition কীভাবে হয়?

```
SCL: ‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾
SDA: ‾‾‾‾‾‾\_____________
     high  ↓low
     (SCL high থাকতে SDA high→low)
```

```
    g = GUARD; while (!(I2C1_SR1 & (1 << 0)) && --g);
```

- `SR1 bit0 = SB` (Start Bit) → START পাঠানো হয়েছে কিনা
- Real life: "রহিমের বেল বাজল কিনা confirm"

```
    I2C1_DR = (addr << 1) | 0;
```

- `DR` = Data Register → hardware এটা bus এ পাঠায়
- `(0x28 << 1) = 0x50` → 7-bit address কে shift
- `| 0` → **LSB = 0 = WRITE** operation
- Real life: **"করিম (0x28), আমি তোমাকে কিছু লিখব (0)"**

Address byte breakdown:

```
Bit:  7 6 5 4 3 2 1 0
Val:  0 1 0 1 0 0 0 0
      └── 0x28 ──┘ ↑
                R/W=0 (write)
```

```
    g = GUARD; while (!(I2C1_SR1 & ((1<<1)|(1<<10))) && --g);
    if (I2C1_SR1 & (1 << 10)) { I2C1_SR1 &= ~(1<<10); I2C1_CR1 |= (1<<9); return 0;
}
```

- `SR1 bit1 = ADDR` → address match হয়েছে কিনা
- `SR1 bit10 = AF` (Acknowledge Failure) → slave ACK দেয়নি?
- `CR1 bit9 = STOP` → STOP দিয়ে bail out
- Real life: "করিম উত্তর দিল? না দিলে বেল বন্ধ করে দিই"

```
    if (!(I2C1_SR1 & (1 << 1))) { I2C1_CR1 |= (1<<9); return 0; }
```

- ADDR set না হলে (timeout / no ACK) → STOP + return 0
- Real life: "কেউ উত্তর দেয়নি, চলে যাই"

```
(void)I2C1_SR1; (void)I2C1_SR2;
```

- **ADDR clear করার magic trick**
- SR1 পড়ো, তারপর SR2 পড়ো → hardware ADDR auto-clear করে
- **Real life:** "করিম উত্তর দিয়েছে, এখন কথা বলি"
- ⚠️ এই step skip করলে I2C bus stuck হয়ে যায়!

```
g = GUARD; while (!(I2C1_SR1 & (1 << 7)) && --g);
I2C1_DR = data;
```

- `SR1 bit7 = TXE` (Transmit Empty) → DR খালি
- `I2C1_DR = 'S'` → আসল data পাঠায়
- Real life: **"করিম, আমার button চাপা ('S')"**

```
g = GUARD; while (!(I2C1_SR1 & (1 << 2)) && --g);
I2C1_CR1 |= (1 << 9);
return 1;
```

- `SR1 bit2 = BTF` (Byte Transfer Finished) → byte complete
- `CR1 bit9 = STOP` → STOP condition
- `return 1` → success
- Real life: "করিম data পেয়েছে, বেল বন্ধ, কথা শেষ"

STOP condition:

```
SCL: ‾‾‾‾‾‾‾‾‾‾‾‾‾‾‾
SDA: ‾‾‾‾‾‾‾‾‾‾‾/‾‾‾‾‾
               low ↑high
     (SCL high থাকতে SDA low→high)
```

Master code — `i2c_read(0x28, &r)`

```
static int i2c_read(unsigned char addr, unsigned char *out) {
    I2C1_CR1 |= (1 << 10);   // ACK enable
```

- `CR1 bit10 = ACK` → master ACK দেবে (default)
- Real life: "আমি শুনতে রাজি"

```
g = GUARD; while ((I2C1_SR2 & (1 << 1)) && --g); if (!g) return 0;
I2C1_CR1 |= (1 << 8);   // START
g = GUARD; while (!(I2C1_SR1 & (1 << 0)) && --g);
```

START + SB wait (আগের মতো)

```
I2C1_DR = (addr << 1) | 1;
```

- `| 1` → **LSB=1 = READ** operation
- Real life: **"করিম, তোমার data দাও"**

```
    g = GUARD; while (!(I2C1_SR1 & ((1<<1)|(1<<10))) && --g);
    if (I2C1_SR1 & (1 << 10)) { ... return 0; }
    if (!(I2C1_SR1 & (1 << 1))) { ... return 0; }
    I2C1_CR1 &= ~(1 << 10);   /* NACK the single byte */
```

🔑 Critical step — NACK:

- `ACK off` = NACK → "আর দিও না, ১টা byte যথেষ্ট"
- Real life: "একটা কথা বলো, আর না"

```
(void)I2C1_SR1; (void)I2C1_SR2;
I2C1_CR1 |= (1 << 9);   /* STOP */
```

- ADDR clear + STOP (order important!)
- STOP = "কথা শেষ"

```
g = GUARD; while (!(I2C1_SR1 & (1 << 6)) && --g);
if (!g) return 0;
*out = (unsigned char)I2C1_DR;
return 1;
```

- `SR1 bit6 = RXNE` → data ready
- `DR read` → করিমের পাঠানো byte
- Real life: "করিম যা বলেছে শুনলাম"

Slave code — করিম কীভাবে উত্তর দেয়

```
I2C1_OAR1 = (MY_ADDR << 1) | (1 << 14);
```

- **OAR1 = Own Address Register 1**
- `MY_ADDR << 1 = 0x50` → address উপরের 7 bits
- `1 << 14` → **7-bit mode** (must be 1,否則 কাজ করবে না)
- Real life: "আমার নাম করিম (0x28), আমি এই বেল এ সাড়া দেব"

```
while (!(I2C1_SR1 & (1 << 1)));   /* ADDR flag wait */
```

- `SR1 bit1 = ADDR` → "আমাকে ডাকা হয়েছে"
- Real life: **"রহিম ডাকল?"**

```
unsigned int sr2 = I2C1_SR2;
```

- **ADDR clear + direction check (একই read এ দুই কাজ)**
- `SR2 bit2 = TRA`: `0` = master writing, `1` = master reading
- Real life: "রহিম কী চায় — কিছু লিখছে নাকি পড়তে চায়?"

Case 1: TRA=0 (master writing)

```
if (!(sr2 & (1 << 2))) {
    while (!(I2C1_SR1 & (1 << 6)));         /* RXNE */
```

```
    unsigned char cmd = (unsigned char)I2C1_DR;
```

- `RXNE` = received byte ready
- `cmd` = রহিম যা পাঠিয়েছে ('S' বা 'K')
- Real life: "রহিম কী বলল শুনি"

```
    while (!(I2C1_SR1 & (1 << 4)));          /* STOPF */
    (void)I2C1_SR1; I2C1_CR1 |= (1 << 0);
```

- `SR1 bit4 = STOPF` → STOP condition received
- STOPF clear + PE re-enable
- Real life: "কথা শেষ, আবার শোনার জন্য তৈরি"

```
    if (cmd == 'S') blink(5);
    else if (cmd == 'K') little_blink();
```

- `'S'` = রহিম চাপ দিয়েছে → আমি blink করি
- `'K'` = আমার press দেখা হয়েছে → ছোট ack
- Real life: **"রহিম চাপ দিয়েছে, আমাকে দেখতে বলেছে — blink করি"**

Case 2: TRA=1 (master reading)

```
} else {
    while (!(I2C1_SR1 & (1 << 7)));          /* TXE */
    I2C1_DR = pressed() ? 'B' : 0x00;
```

- **আমার button state পাঠাই**: চাপা → 'B', না → 0x00
- Real life: **"রহিম আমার button কী অবস্থা জিজ্ঞেস করছে, বলি"**

```
    while (!(I2C1_SR1 & (1 << 10)));         /* AF */
    I2C1_SR1 &= ~(1 << 10);
}
```

- `SR1 bit10 = AF` → রহিম NACK দিয়েছে (transfer শেষ)
- Real life: "রহিম বলল 'যথেষ্ট, থামো'"

🎯 I2C Summary — "কখন কী হয়"

Event	Master	Slave
Idle	button check	ADDR wait
Master button press	'S' write	ADDR → RXNE → cmd='S' → blink(5)
Slave button press	periodic read	returns 'B'

Event	Master	Slave
Master sees 'B'	blink(5) + 'K' write	—
Slave sees 'K'	—	little_blink()
No slave	GUARD timeout → return 0	—

🎬 PART 2: UART — Real Life Story

📖 Real Life Setup

দুই বন্ধু — রহিম আর করিম — দুই পাশে দুটো ঘরে। দুইজনের **নিজের কলিং বেল (TX)** আর **নিজের শ্রবণযন্ত্র (RX)** আছে। দুইজনের বেল আর শ্রবণযন্ত্র ক্রস-কানেক্টেড।

নিয়ম (UART Protocol):

- **কোনো shared clock নেই** — দুইজন নিজের গতিতে কথা বলে, কিন্তু **একই speed** এ (baud rate = 9600)
- **যেকোনো কেউ কথা বলতে পারে** (symmetric)
- **দুজন একসাথে কথা বলতে পারে** (full-duplex!) — আলাদা বেল/শ্রবণ
- **কেউ "master" না, কেউ "slave" না** — peer-to-peer
- **একই code দুইজনের ঘরে** — symmetrical

🎭 Real Life Scenario 1: রহিমের button চাপা

🎬 ঘটনার ক্রম

T=0ms: রহিম button চাপে

```
রহিমের ঘর:                    করিমের ঘর:
🔘 Button চাপা                 🔘 Button চাপা না
PC13 = LOW                     PC13 = HIGH
💡 LED off                     💡 LED off
```

T=0-20ms: রহিম debounce করে

```
int now = pressed();     // 1
if (now && !prev) {      // rising edge
    delay_ms(20);        // debounce
    if (pressed()) {     // still pressed?
```

T=20ms: রহিম `send('S')` করে

```
রহিমের code:
  USART1_DR = 'S'  → TX wire এ bit-by-bit যায়
  awaiting = 1     → "আমি ACK এর অপেক্ষায়"
  timer = 6000     → 6 sec timeout

UART frame on TX wire:
  START bit (LOW) → 8 data bits → STOP bit (HIGH)
```

T=20-21ms: করিম receive করে

```
করিমের code:
  RXNE flag set
  c = 'S' পড়ল
  c=='S' → blink(5) শুরু
         → send('D')  (ack)
```

করিম ভাবে: "রহিম চাপ দিয়েছে, আমি blink করি, তারপর ack পাঠাই"

T=21-2021ms: করিমের LED ৫বার blink

```
T=21:   ON  → 200ms
T=221:  OFF → 200ms
T=421:  ON  → 200ms
...
T=2021: blink complete
```

T=21ms: করিম send('D') করেছে

```
UART frame on করিমের TX (রহিমের RX):
  'D' = 0x44 = 0100 0100
  START → 0100 0100 → STOP
```

T=22ms: রহিম 'D' পায়

```
রহিমের code:
  RXNE flag set
  c = 'D' পড়ল
  c=='D' && awaiting==1 → true!
  → little_blink() (2×120ms)
  → awaiting = 0
```

রহিম ভাবে: "করিম ack দিয়েছে, আমি 2 blink দিয়ে confirm করি"

T=22-502ms: রহিমের LED ২বার ছোট blink

```
T=22:   ON  → 120ms
T=142:  OFF → 120ms
T=262:  ON  → 120ms
T=382:  OFF → 120ms
```

🏁 Final Result

```
রহিম (Board A):      করিম (Board B):
🔘 button: চাপা ছিল 🔘 button: চাপা না
💡 LED: 2 blink ছোট 💡 LED: 5 blink বড়
(sending board এর ack)  (receiving board এর reaction)
```

🎭 Real Life Scenario 2: করিমের button চাপা

🎬 ঘটনার ক্রম

T=0ms: করিম button চাপে

```
করিমের ঘর:                    রহিমের ঘর:
🔘 Button চাপা                 🔘 Button চাপা না
💡 LED off                     💡 LED off
```

T=0-20ms: করিম debounce করে (একই code!)

```
int now = pressed();    // 1 (করিমের)
if (now && !prev) { ... }
```

T=20ms: করিম `send('S')` করে

```
করিম ভাবে: "আমার button চাপা, রহিম কে বলি"
send('S') → রহিমের RX এ যায়
awaiting = 1
timer = 6000
```

T=21ms: রহিম 'S' পায়

```
রহিমের code:
  c = 'S'
  c=='S' → blink(5) শুরু
         → send('D')  (ack)

রহিম ভাবে: "করিম চাপ দিয়েছে, আমি blink করি, তারপর ack পাঠাই"
```

T=21-2021ms: রহিমের LED ৫বার blink

```
রহিমের LED: ON/OFF ×5
```

T=2022ms: করিম 'D' পায়

```
করিমের code:
  c = 'D', awaiting==1 → true!
  → little_blink()
  → awaiting = 0
```

T=2022-2502ms: করিমের LED ২বার ছোট blink

🏁 Final Result

```
রহিম (Board A):      করিম (Board B):
🔘 button: চাপা না   🔘 button: চাপা ছিল
💡 LED: 5 blink বড়  💡 LED: 2 blink ছোট
```

মজার ব্যাপার: I2C তে slave passively poll হয়, UART এ **যে চাপে সে নিজেই message পাঠায়** — fully symmetric!

💻 Code — Line by Line

`send()` — UART transmit

```
static void send(char c) {
    while (!(USART1_SR & (1 << 7)));
    USART1_DR = c;
}
```

Line 1: `SR bit7 = TXE` (Transmit Empty) → TX buffer empty?

- Real life: **"রহিমের কলিং বেল কি খালি?"**
- `!TXE` (buffer full) → wait (আগের byte পাঠানো হয়নি)

Line 2: `DR = c` → hardware TX pin দিয়ে bit-by-bit পাঠায়

Real life: **"বেল বাজাও"**

`main()` receive logic

```
if (USART1_SR & (1 << 5)) {
    char c = (char)USART1_DR;
```

Line 1: `SR bit5 = RXNE` (Receive Not Empty) → data এসেছে?

Real life: **"করিমের বেল শুনলাম?"**

Line 2: `c = DR` → data read (auto-clear RXNE)

Real life: "কী বলল শুনি"

```
    if (c == 'S') {
        blink(5);
        send('D');
    } else if (c == 'D' && awaiting) {
        little_blink();
        awaiting = 0;
    }
```

`c == 'S'`: other board চাপ দিয়েছে

- `blink(5)` → আমার LED ৫বার (reaction)
- `send('D')` → "দেখেছি" ACK

`c == 'D' && awaiting`: my press acked

- `awaiting` check জরুরি — stray 'D' ignore
- `little_blink()` → ২বার ছোট
- `awaiting = 0` → "আর অপেক্ষা নেই"

Main loop state machine

```
int prev = 0, awaiting = 0;
unsigned int timer = 0;
```

- `prev` = আগের button state (rising edge detect)
- `awaiting` = "আমি 'S' পাঠিয়েছি, 'D' এর অপেক্ষায়"
- `timer` = timeout counter

```
int now = pressed();
if (now && !prev) {
    delay_ms(20);
    if (pressed()) { send('S'); awaiting = 1; timer = ACK_TIMEOUT; }
}
prev = now;
```

- **Rising edge:** `now && !prev`
- **Debounce:** `delay_ms(20)` + re-check
- **Send + set state:** `send('S')`, `awaiting=1`, `timer=6000`
- **State update:** `prev = now`

```
if (awaiting && --timer == 0) awaiting = 0;
```

- 6 sec এর মধ্যে 'D' না এলে → give up
- Real life: "করিম ৬ সেকেন্ডেও উত্তর দিল না, ছেড়ে দিই"

🎯 UART — Full Duplex Real Life

রহিমের TX ——————————► করিমের RX
রহিমের RX ◄—————————— করিমের TX

একসাথে দুইজন কথা বলতে পারে! (এই code এ polling-based, তাই practically একটা block হলেও hardware full-duplex capable)

UART Frame (8N1):

```
Idle(HIGH) ──┐     ┌────────┐     ┌── Idle(HIGH)
             │     │ 8 bit  │     │
             └─────┤ data   ├─────┘
              START ↑     ↑ STOP
                    D0..D7

' S ' = 0x53 = 0101 0011
TX line: H ──L─0─1─0─1─0─0─1─1─H──
            |S| D0..D7      |STOP|
            10 bits total = 1 start + 8 data + 1 stop
```

🎬 PART 3: SPI — Real Life Story

📖 Real Life Setup

রহিম (Master) আর করিম (Slave) দুই ঘরে। এবার একটা **shared ঘড়ি (SCK)** — শুধু রহিম বাজায়। দুইজনের **বেল (MOSI = রহিম→করিম)** আর **শ্রবণ (MISO = করিম→রহিম)**। আর একটা **NSS switch** (রহিম control করে)।

নিয়ম (SPI Protocol):

- **রহিম ঘড়ি বাজায়** (master clock দেয়)
- **রহিমের বেল (MOSI)** + **করিমের বেল (MISO)** আলাদা wire
- **রহিম NSS LOW করলে** করিম "selected"
- **প্রতি ঘড়ির tick এ একইসাথে দুই দিকের 1 bit যায়** (full-duplex!)
- **একটা transfer = একইসাথে send + receive**

🎭 Real Life Scenario 1: রহিমের button চাপা

🎬 ঘটনার ক্রম

T=0ms: রহিম button চাপে

```
রহিমের ঘর:                  করিমের ঘর:
🔘 Button চাপা               🔘 Button চাপা না
PC13 = LOW                   PC13 = HIGH
💡 LED off                   💡 LED off
```

```
করিমের buffer: 0x00 (preloaded)
```

T=0-20ms: রহিম debounce করে

T=20ms: রহিম `spi_xfer('S')` করে

```
spi_xfer('S') internal:
  1. TXE wait (buffer খালি)
  2. SPI2_DR = 'S'
  3. Hardware: 8 SCK pulse generate
     প্রতিটা pulse এ:
       MOSI bit → করিমের shift register এ
       MISO bit ← করিমের preloaded byte থেকে
  4. RXNE wait
  5. return DR  (করিমের পাঠানো byte)

Real life:
  রহিম ঘড়ি বাজায় ৮বার
  প্রতিটা tick এ:
    রহিমের 'S' এর 1 bit → করিমের ঘরে
    করিমের 0x00 এর 1 bit → রহিমের ঘরে
  ৮ tick শেষে:
    করিমের কাছে: 'S'
    রহিমের কাছে: 0x00 (করিমের আগের preload)
```

T=21ms: করিম `'S'` পায়

```
করিমের code:
  RXNE flag set
  sr = SPI2_SR (check TXE)
  cmd = SPI2_DR = 'S' পড়ল
  TXE check → 1 (buffer খালি ছিল)
  reply = pressed() ? 'B' : 0x00  // e.g. 0x00
  cmd == 'S' → blink(5) শুরু, reply = 'D'
  preload: SPI2_DR = 'D' (পরের transfer এ যাবে)
```

T=21ms: রহিম `delay(2500)` শুরু

```
রহিম ভাবে: "করিম blink শেষ করুক"
রহিমের LED off
```

T=21-2021ms: করিমের LED ৫বার blink

T=2521ms: রহিম আবার `spi_xfer(0x00)` করে

```
spi_xfer(0x00) internal:
  TXE wait
  SPI2_DR = 0x00 (dummy)
  8 SCK pulse:
    রহিমের 0x00 → করিম
    করিমের preload 'D' → রহিমের
  RXNE wait
  return 'D'

Real life:
  রহিম ৮বার ঘড়ি বাজায়
  করিমের preload 'D' রহিমের ঘরে আসে
```

T=2522ms: রহিম `'D'` পায়

রহিমের code:
  r = spi_xfer(0x00) = 'D'
  r == 'D' → little_blink()

T=2522-3002ms: রহিমের LED ২বার ছোট blink

🏁 Final Result

রহিম (MASTER): করিম (SLAVE):
🔘 button: চাপা ছিল 🔘 button: চাপা না
💡 LED: 2 blink ছোট 💡 LED: 5 blink বড়
(ack) (response)

🎭 Real Life Scenario 2: করিমের button চাপা

🎬 ঘটনার ক্রম

T=0ms: করিম button চাপে

করিমের ঘর:
🔘 Button চাপা
PC13 = LOW
💡 LED off

রহিমের ঘর:
🔘 Button চাপা না
PC13 = HIGH
💡 LED off

করিমের buffer: 0x00 (preloaded, কিন্তু button এখুনি চাপা)

T=10ms: রহিম polling করে

রহিমের code:
  r = spi_xfer(0x00) (dummy)
  8 SCK pulse:
    রহিমের 0x00 → করিম
    করিমের preload (0x00, পুরনো) → রহিম
  return 0x00 → r != 'B' → ignore

⚠️ এক delay!
রহিম জানল না এখনই, কারণ করিমের buffer এ পুরনো 0x00 ছিল

T=11ms: করিম receive করে

করিমের code:
  RXNE, cmd = 0x00 (রহিমের dummy)
  sr check: TXE=1 (buffer খালি)
  reply = pressed() ? 'B' : 0x00 → 'B' (এখন চাপা!)
  cmd == 0x00, unknown না
  preload: SPI2_DR = 'B'

করিম ভাবে: "রহিম কিছু পাঠাল, আমি 'B' preload করি"

T=21ms: রহিম আবার poll করে

```
রহিমের code:
  r = spi_xfer(0x00)
  8 SCK pulse:
    রহিমের 0x00 → করিম
    করিমের preload 'B' → রহিম
  return 'B'  ✅

  nowB = (r == 'B') = 1
  prevB = 0
  nowB && !prevB → rising edge!
  blink(5) শুরু
  spi_xfer('K')  → করিম কে
  delay(600)
```

T=21-2021ms: রহিমের LED ৫বার blink

T=21ms: রহিম `'K'` পাঠায়, করিম পায়

```
করিমের code:
  cmd = 'K'
  cmd=='K' → little_blink() শুরু
  preload: SPI2_DR = pressed() ? 'B' : 0x00
```

T=21-501ms: করিমের LED ২বার ছোট blink

🏁 Final Result

```
রহিম (MASTER):      করিম (SLAVE):
🔘 button: চাপা না   🔘 button: চাপা ছিল
💡 LED: 5 blink বড়  💡 LED: 2 blink ছোট
(পোলিং করে জেনেছে)  (ack)
```

🔑 **SPI এর key behavior: One-byte delay!** কারণ:

- Master `spi_xfer(0x00)` পাঠায় → reply আগের preload
- Slave এখন নতুন reply preload করে
- পরের transfer এ সেটা আসে

💻 Code — Line by Line

`spi_xfer()` — SPI heart

```
static unsigned char spi_xfer(unsigned char b) {
```

Purpose: "এক byte পাঠাও, এক byte নাও" — same time!

```
    while (!(SPI2_SR & (1 << 1)));
```

- `SR bit1 = TXE` (Transmit Empty) → TX buffer empty?

- Real life: **"আমার বেল টেবিলে আগের byte আছে? খালি হলে পাঠাই"**

```
SPI2_DR = b;
```

- DR এ write → hardware এটা shift register এ load করে
- **8 SCK pulse generate** হয় automatically
- প্রতিটা pulse এ:
  - MOSI দিয়ে 1 bit যাচ্ছে
  - MISO দিয়ে 1 bit আসছে
- Real life: **"রহিম ঘড়ি বাজাচ্ছে, দুই দিকেই bit যাচ্ছে"**

```
while (!(SPI2_SR & (1 << 0)));
```

- `SR bit0 = RXNE` (Receive Not Empty) → RX buffer received?
- Real life: **"করিম কিছু পাঠিয়েছে?"**

```
return (unsigned char)SPI2_DR;
```

- DR read → করিমের byte
- Real life: **"করিম কী পাঠাল শুনি"**

Master init — `SPI2_CR1` magic line

```
SPI2_CR1 = (1 << 2) | (5 << 3) | (1 << 9) | (1 << 8) | (1 << 6);
```

Bit by bit:

- `1 << 2` → **MSTR=1** (Master mode)
- `5 << 3` → **BR=101** → baud divider = /64 → 16MHz/64 = 250kHz
- `1 << 9` → **SSM=1** (Software Slave Management)
- `1 << 8` → **SSI=1** (Internal NSS HIGH = "selected")
- `1 << 6` → **SPE=1** (SPI Enable)

Real life: **"আমি রহিম, master; ঘড়ি 250kHz এ বাজাব; internal switch দিয়ে নিজেকে selected রাখি"**

Master main loop — Scenario 1

```
spi_xfer('S');
```

- 'S' পাঠাল, করিম পাবে
- Return value (করিমের পুরনো byte) **ignore** — "এখনই reply আসবে না"

```
delay_ms(SLAVE_BLINK_WAIT);   // 2500ms
```

- করিম blink(5) = 2000ms, wait 2500ms
- Real life: "করিম blink শেষ করুক"

```
if (spi_xfer(0x00) == 'D')
    little_blink();
```

- **Dummy 0x00 পাঠাল** → SCK pulse generate
- করিমের preload 'D' ফেরত আসে
- 'D' == 'D' → little_blink
- Real life: "করিম বলল 'দেখেছি' (D), আমি ack দিই"

Master main loop — Scenario 2 (polling)

```
unsigned char r = spi_xfer(0x00);
int nowB = (r == 'B');
if (nowB && !prevB) {
    blink(5);
    spi_xfer('K');
    delay_ms(SLAVE_LITTLE_WAIT);
}
prevB = nowB;
```

- `spi_xfer(0x00)` → dummy পাঠায়, করিমের reply পড়ে
- `r == 'B'` → করিম pressed
- `nowB && !prevB` → rising edge
- `blink(5)` → আমার reaction
- `spi_xfer('K')` → করিম কে "দেখেছি"
- `delay(600)` → করিমের little-blink শেষ হওয়া পর্যন্ত wait
- Real life: **"করিম চাপ দিয়েছে, আমি blink করি, তারপর ack পাঠাই"**

Slave init — critical differences

```
static void spi_reset(void) {
    RCC_APB1RSTR |=  (1 << 14);
    RCC_APB1RSTR &= ~(1 << 14);
    SPI2_CR1 = (1 << 9) | (1 << 6);
}
```

Line 1-2: SPI2 hardware reset (RSTR toggle)

Real life: **"পুরো সিস্টেম রিসেট"**

Line 3: `SPI2_CR1 = (1<<9)|(1<<6)`

- **MSTR=0** (Slave) — master এর থেকে main difference!
- **SSM=1**, **SPE=1**
- `SSI=0` (slave — master select করবে)

Real life: **"আমি করিম, slave; রহিমের ঘড়ি follow করি"**

Slave preload (critical!)

```
while (!(SPI2_SR & (1 << 1)));   // TXE
SPI2_DR = 0x00;                  // preload
```

- **First transfer এ slave এর shift register এ কিছু থাকতে হবে**
- না থাকলে garbage পাঠাবে
- Real life: **"আমার বেল এ প্রথম থেকেই 'button চাপা না' রাখি"**

Slave main loop

```
while (!(SPI2_SR & (1 << 0)));        // RXNE wait
unsigned int  sr  = SPI2_SR;
unsigned char cmd = (unsigned char)SPI2_DR;
```

- `RXNE` wait → master transfer করেছে
- `sr` = full status snapshot (TXE check এর জন্য)
- `cmd` = master এর পাঠানো byte

```
if (!(sr & (1 << 1))) spi_reset();
```

🔑 **Critical:**

- `sr bit1 = TXE=0` → আমার reply এখনো buffer এ (পাঠানো হয়নি)
- মানে **master আগেই পরের byte পাঠিয়েছে** → আমি এক byte পিছিয়ে
- **Reset** করি fresh start এর জন্য
- Real life: "আমি উত্তর দিতে দেরি করেছি, রহিম আরেকবার বলেছে — reset করি"

```
unsigned char reply = pressed() ? 'B' : 0x00;
```

- Default reply = আমার button state
- Real life: **"আমার button চাপা হলে 'B' বলি, না হলে '0'"**

```
if      (cmd == 'S') { blink(5); reply = 'D'; }
else if (cmd == 'K') little_blink();
else if (cmd != 0x00) { delay_ms(1); spi_reset(); }
```

`cmd == 'S'`:

- রহিম চাপ দিয়েছে → আমি blink(5)
- reply = 'D' (ACK)
- Real life: **"রহিম চাপ দিয়েছে, আমি blink করি, তারপর 'D' (দেখেছি) বলি"**

`cmd == 'K'`:

- রহিম আমার press acknowledge করেছে
- little_blink (2×120ms)
- Real life: **"আমার press রহিম দেখেছে, আমি ছোট ack দিই"**

`cmd != 0x00` (unknown):

- Bit sync lost (unknown byte এসেছে)
- `delay_ms(1)` দিয়ে settle, `spi_reset()` fresh start
- Real life: "কিছু ভুল হয়েছে, রিসেট করি"

```
while (!(SPI2_SR & (1 << 1)));   // TXE
SPI2_DR = reply;
```

- পরের transfer এর জন্য reply preload
- Real life: **"আমার পরের উত্তর তৈরি রাখি"**

🎯 SPI Summary — "কখন কী হয়"

Event	Master	Slave
Idle	button check + polling	RXNE wait (preloaded 0x00)
Master button press	`'S'` send → delay(2500) → dummy পাঠায় `'D'` collect	`'S'` receives → blink(5) → preload `'D'`
Slave button press	poll → `'B'` receives	preload `'B'`, পরে transfer এ যায়
Master sees `'B'`	blink(5) + `'K'` sends	`'K'` receives → little_blink
Wrong/unknown byte	—	`delay(1)` + `spi_reset()`
Slave absent	MISO pull-down → 0x00 আসে	—

📊 তিনটা Protocol — Scenario Comparison Table

Scenario: Master এর button চাপা

Step	I2C	UART	SPI
Send	`i2c_write('S')` (address+data)	`send('S')` (just byte)	`spi_xfer('S')` (byte out)

Step	I2C	UART	SPI
Slave receives	ADDR flag → TRA=0 → RXNE → 'S'	RXNE → 'S'	RXNE → 'S'
Slave reaction	blink(5)	blink(5) + send('D')	blink(5) + preload 'D'
Master gets ack	❌ no explicit ACK for 'S'	✅ 'D' arrives (RXNE)	✅ dummy পাঠিয়ে 'D' collect
Master reaction	delay(2500) → little_blink	'D' receive → little_blink	'D' receive → little_blink
Total time	~2700ms	~2500ms	~3000ms
Bus driver	Master only	Either	Master only

Scenario: Slave এর button চাপা

Step	I2C	UART	SPI
Slave noticed	❌ cannot initiate	✅ send('S')	❌ cannot initiate
Master noticed	polls `i2c_read()` → 'B'	receives 'S' → blink(5)	polls `spi_xfer(0x00)` → 'B'
Master reaction	blink(5) + `i2c_write('K')`	blink(5) + `send('D')`	blink(5) + `spi_xfer('K')`
Slave ack	'K' → little_blink	'D' → little_blink	'K' → little_blink
Symmetry	❌ Master-slave	✅ Both can initiate	❌ Master-slave

🎓 Real Life Analogy — Final Summary

I2C — "Teacher আর Student"

```
👨‍🏫 Teacher (Master): জিজ্ঞেস করে, answer নেয়, control করে
👦 Student (Slave):   শুধু teacher এর প্রশ্নের উত্তর দেয়

- Teacher: "করিম, তোমার button কী?" → Student: "চাপা (B)"
```

- Teacher: "সবাই শোনো, আমার button চাপা (S)" → Student: blink
- Student নিজে কিছু বলতে পারে না!

Use case: এক teacher, অনেক student → address দিয়ে চেনা

UART — "Two Friends Chatting"

👫 রহিম আর করিম: Peer-to-peer
- দুজনই কথা বলতে পারে
- দুজনই একসাথে কথা বলতে পারে (full-duplex)
- কোনো "teacher" নেই
- same code দুইজনের কাছে

Use case: ২টা device এর মধ্যে direct chat (PC ↔ MCU, GPS module)

SPI — "Master with Many Servants (but fast)"

👑 Master: ঘড়ি বাজায়, select করে, command দেয়
👨‍🍳 Servant: শুধু selected হলে respond করে

- প্রতি clock tick এ একইসাথে data যায় আর আসে
- Master: "S" (command) + Servant: "B" (reply) — same time!
- Master ছাড়া servant কিছু করতে পারে না

Use case: Fast data, few devices (SD card, display, Flash)

📋 Exam এ এই Table গুলো লিখো

Protocol Selection Quick Table

Situation	Protocol	কারণ
Many sensors, few pins	**I2C**	Address-based
Fast bulk data (SD, LCD)	**SPI**	MHz speed
PC ↔ MCU	**UART**	Standard COM port
Long distance	**UART**	RS-485 possible
2 devices, simple	**UART**	Symmetric code
Address required	**I2C**	7-bit address
Full duplex, fast	**SPI**	Separate MOSI/MISO

Situation	Protocol	কারণ
Multiple slaves	**I2C** (if many) / **SPI** (if few)	Pin count

Packet Format Summary

I2C (per byte):

```
START | addr(7) + R/W(1) | ACK | data(8) | ACK | ... | STOP
```

UART (per byte, 8N1):

```
START(1) | data(8) | STOP(1)  = 10 bits
```

SPI (per byte):

```
NSS=LOW | 8 SCK pulses | MOSI+MISO simultaneous | NSS=HIGH
```

Button → LED Behavior (Final Reference)

Press	I2C	UART	SPI
Master presses	Master: 2 blink (ack) / Slave: 5 blink	Both: same code, 5 blink on receiver, 2 on sender	Master: 2 blink / Slave: 5 blink
Slave presses	Master polls → 5 blink, sends 'K' → Slave: 2 blink	Slave sends 'S' → Master: 5 blink → 'D' → Slave: 2 blink	Master polls → 5 blink, sends 'K' → Slave: 2 blink
Both idle	Both dark	Both dark	Both dark
Slave absent	GUARD timeout (re-turn 0)	awaiting timeout (6s)	MISO pull-down (0x00)
Recovery	STOP + return	Timer expire	SPI reset

🎯 Final Memory Mantra (Exam এর আগে 1 min)

```
I2C:  2 wire, address, "Teacher-Student", half-duplex
      Master controls everything, Slave passive

UART: 2 wire, no address, "Two friends", full-duplex
      Symmetric code, either can initiate
```

```
SPI:  3+ wire, no address (NSS), "Master-Servant", full-duplex
      Master generates clock, Slave follows, one-byte delay
```

Packet format mantra:

```
I2C:  START + Addr + R/W + Data + ACK + STOP
UART: START(1) + Data(8) + STOP(1) = 10 bits per byte
SPI:  8 SCK pulses = 1 byte in each direction simultaneously
```

শুভকামনা exam এর জন্য! 🎓

কোনো specific scenario আরও practice করতে চাইলে বলো — আমি mock প্রশ্ন দিয়ে practice করাবো। **মনে রাখো: এই তিনটা protocol এর পার্থক্য মুখস্থ না, বুঝে মনে রাখো — "কে clock দেয়, কে initiate করে, কত wire লাগে, কত fast" — এই ৪টা প্রশ্নের উত্তর দিলেই যেকোনো exam প্রশ্নের উত্তর দিতে পারবে।** 💪